

Better Ways to Build Websites That Work



HTML & CSS The Good Parts

HTML & CSS: The Good Parts

HTML and CSS are the workhorses of web design, and using them together to build consistent, reliable web pages requires both skill and knowledge. The task is more difficult if you're relying on outdated, confusing, and unnecessary HTML hacks and workarounds. Author Ben Henick shows you how to avoid those traps by going beyond the standard tips, tricks, and techniques to connect the underlying theory and design of HTML and CSS to your everyday work habits.

With this practical book, you'll learn how to work with these tools far more effectively than is standard practice for most web developers. Whether you handcraft individual pages or build templates, *HTML & CSS: The Good Parts* will help you get the most out of these tools in all aspects of web page design—from layout to typography to color.

- Structure HTML markup to maximize the power of CSS
- Implement complex multicolumn layouts from scratch
- Improve site production values with advanced CSS techniques
- Support formal usability and accessibility requirements with tools built into HTML and CSS
- Avoid the most annoying browser and platform limitations

“Ben gives you all the tools you need to create powerfully flexible markup, bringing Zen to web development and the workplace.”

—Michael Scafidi
Presentation Layer
Technology Director,
Razorfish

Ben Henick has been producing websites since 1995. His clients include small businesses as well as companies that are household names. Ben has written about web development for alistapart.com, digital-web.com, and the Opera Software Web Standards Curriculum, among others.

Praise for *HTML & CSS: The Good Parts*

“Ben has an encyclopedic knowledge of web development and makes even the most obtuse-sounding concepts seem eminently approachable. All while writing a book filled with charm, wit, and aplomb. (Yeah, I hate him, too. Great book, though.)”

— Ethan Marcotte, coauthor of *Designing with Web Standards*,
Third Edition

“*HTML & CSS: The Good Parts* is essential for those who work building web pages and need to take their understanding and knowledge to the next level. Web developers and designers of all types need to have solid depth of understanding of how HTML and CSS work as well as how they interact with the browser. The difference I find between an okay web designer and developer (including those who work with tools that create and manage sites) and a really good one is the depth of understanding they have and use of HTML and CSS. This book provides that depth and understanding.

“In my opinion one of the best pieces for me in this book is the inclusion of the proper structuring of pages, sites, and the depth of the discussion for integration is essential for the maintenance, use, and even SEO considerations. This is something that far too often gets missed and is not understood well. Having this knowledge and these skills in your tool belt will only lead to much improved outcomes that are easier to build out, manage, and use.”

— Thomas Vander Wal, founder and senior consultant at
InfoCloud Solutions

“I’ve always said that the beauty (and the frustration) in CSS is that there are so many ways to do things. Ben has done a fantastic job of homing in on the good, the bad, and the ugly in the broad CSS realm. His useful real-world approach not only gives you a great reference to the most commonly used elements, properties, and values, but it also addresses the advantages (and pitfalls) of various techniques. Whether you’re working on small or large sites, Ben clearly presents the principles you need to crank your skills up to the next level.”

— Stephanie Sullivan, author, *Mastering CSS with Dreamweaver CS4*

Ben Henick

~~Beijing Cambridge Farnham Köln Sebastopol Taipei Tokyo~~ **HTML** **& CSS: The Good Parts**

HTML & CSS: The Good Parts

by Ben Henick

Copyright © 2010 Ben Henick. All rights reserved.
Printed in the United States of America.

Published by O'Reilly Media, Inc., 1005 Gravenstein Highway North, Sebastopol, CA 95472.

O'Reilly books may be purchased for educational, business, or sales promotional use. Online editions are also available for most titles (<http://my.safaribooksonline.com>). For more information, contact our corporate/institutional sales department: 800-998-9938 or corporate@oreilly.com.

February 2010: First Edition.

Editor: Simon St.Laurent
Production Editor: Loranah
Dimant **Copyeditor:** Emily
Quill
Proofreader: Sada Preisch

Indexer: Lucie Haskins
Cover Designer: Karen
Montgomery **Interior**
Designer: David Futato
Illustrator: Robert Romano

Printing History:

Nutshell Handbook, the Nutshell Handbook logo, and the O'Reilly logo are registered trademarks of O'Reilly Media, Inc. *HTML & CSS: The Good Parts*, the image of a ring-tailed cat, and related trade dress are trademarks of O'Reilly Media, Inc.

Many of the designations used by manufacturers and sellers to distinguish their products are claimed as trademarks. Where those designations appear in this book, and O'Reilly Media, Inc., was aware of a trademark claim, the designations have been printed in caps or initial caps.

While every precaution has been taken in the preparation of this book, the publisher and author assume no responsibility for errors or omissions, or for damages resulting from the use of the information contained herein.

ISBN: 978-0-596-15760-9

[SB]

1266416276

*To the memory of my mother and the
patience of my father—each a wellspring of
love, hope, and knowledge.*

Table of Contents

Preface	xvii
----------------------	-------------

1. Hypertext at the Core	
---------------------------------------	--

.....	1	The Web Without Links	1	URIs	2
		Managing Links	3	Improving the User Experience with Linking	3
		Hypertext Implementation Challenges	4		

2. Working with HTML Markup	
--	--

.....	7	HTML Syntax	7	Tags, Elements, and Attributes	8
		Page Structure	10	Rendering Modes, Flavors of HTML, and Document Type Declarations	10
		HTML or XHTML?	11	Strict, Transitional, or Frameset?	12
		A Tale of Two Box Models	12	Choosing the Right Document Type for Your Project	13
		Beautiful Parts: Universal Attributes	14	Providing Stylesheet Hooks with class and id	14
		Describing Content with title and lang	15	The contenteditable Attribute in HTML5	17
		Separating Content, Structure, Presentation, and Behavior	18	Making Your Sites “Safe As Houses”	18
		Separation in Practice	18	Working with Document Trees	19
		Browsers, Parsing, and Rendering	20	Dynamic HTML, Ajax, and Rendering	21

3. CSS Overview	
------------------------------	--

.....	23	Connecting Stylesheets to HTML Documents	23
-------	-----------	--	----

vii

		Referencing a Stylesheet with link	23	Targeting Internet Explorer Versions with Conditional Comments	24
		Replacing link with style	25	Using @import	25
		Beware of style Attributes!	25	Targeting Rules to Specific Media	26
		Choosing the Elements You Want to Style: Writing Selectors	27	Parents, Children, and Siblings: Element/Node Relationships	28
		Simple			

Selectors 29 Multiple and Descendant Selectors 29 Selecting Direct Child Elements 30	
Rule Conflicts, Priority, and Precedence 31 Selector Priority 31 Avoiding Rule Conflicts 32 Value Inheritance 33	
CSS Property and Value Survey 33 CSS Units 33 Cross-Media Length and Size Units 34 Pitch and the Value of a Pixel 34 Print-Friendly Length Units 36 font-size Keywords 36 Color Units 37 Key CSS Layout Properties 37	

4. Developing a Healthy Relationship with Standards

. 41 The Broad Landscape of Web-Related Standards 41 Why Web Standards? 42	
Interoperability 42 Market Forces 43 Forward Compatibility 43 Accessibility 43 Vendor Priorities 44 Legacy Asset Inertia 44 Best Practices (and Lack Thereof) 44 Strict Constructionism 45	
Taking the Middle Road: Standards-Friendliness 45 Benefits of Standards-Friendliness 46 Rules of Standards-Friendly Development 46	

5. Effective Style and Structure

. 49 The Four Habits of Effective Stylists 49	
Habit #1: Keeping It Simple 50 Habit #2: Keeping It Flexible 52	

viii | Table of Contents

Habit #3: Keeping to Consistency 55 Habit #4: Keeping Your Bearings 57	
CSS Zen and the Stylist's Experience 59 The Functional Principles of CSS Zen 60 Information Architecture and Web Usability 61	
Multidimensionality 62 Navigation: Orientation and Wayfinding 63 Visit Strategies 64 Guideposts for Creating Usable Interfaces 66 Predicting Visitor Behavior with Scenarios and User Testing 67 Taxonomy and Nomenclature 68 Applying Taxonomy Through the Cascade 70 New Structural Elements (HTML5) 72	

6. Solving the Puzzle of CSS Layout

. 73 The CSS Box Model and Element Size Control 73 Quirks Mode and Strict Mode 73 auto Values 74	
The overflow Property 75 Limiting But Not Fixing Element Dimensions 77 Handling the Unpredictable 77	
Margins, Borders, and Padding 78 Negative Margins 79 Collapsed Margins 80 Borders 81 Padding 82 The Box Behavior of the Document Root Elements 82 Box Property Dimensions and the % Value 82	
Element Flow 83 Inline Elements 83 Block Elements 83 Inline-Block Elements 84	
Using the display Property to Change an Element's Flow 84 The display Property 85 The float and clear Properties 86 The Rules of the float Property 86 Canceling float Values with Corresponding clear Values 87	

float Context	88
Implementing Multicolumn Layouts	88
Converting the Two-Column Layout from Markup Tables to CSS	89
How the Two-Column Styles Work	90
Benefits of Confining Layout Specifications to Stylesheets	92
Moving from Two Columns to Three	93

Table of Contents | ix

Dealing with More Than Three Columns	95
Semantically Empty Containers for Multicolumn Layouts	95
Advanced Layout in CSS3	96
CSS Positioning Properties	96
How Positioning Works	96
Bounding Positioned Elements	99
The visibility and z-index Properties	99
Altering Visibility Without Affecting Document Flow	100
Stacking	101
Obtaining Precise Navigation Source Order and Layout	102
Orienting the List	102
Forcing the Navigation List into the Desired Coordinates	104
Layout Types and Canvas Grids	106
Fixed, Proportional, and Flexible Layouts	106
Defining Grids	108
The Rule of Thirds, the Golden Ratio, and the Fibonacci Sequence	110
Implementing a Flexible Page Grid	111

7. Working with Lists

115	Ordered and Unordered Lists	115
User Agent Default Styles for Ordered and Unordered Lists	115	
Creating Valid Ordered and Unordered Lists	116	
The list-style-type Property and the type Attribute	116	
The nav Element (HTML5)	117	
Changing the Range of an Ordered List	119	
Other Uses for Lists	120	
Outlines	120	
Inline Serial Lists	120	
Altering the Layout of Footer Links	121	
Bullets in Backgrounds?	121	
Styling Navigation Elements	121	
Placing the Primary Site Navigation Within the Source Order	122	
The Primary Navigation Layout Recipe	122	
The Footer Navigation Recipe	123	
Definition Lists	124	
Styling Definition Lists	124	
Dictionary Example	125	
Dialogue Example	127	

8. Headings, Hyperlinks, Inline Elements, and Quotations . . .

129	Headings and Good Writing	129
Headings in Print	129	
Optimal Heading Insertion	131	

x | Table of Contents

Styling Heading Elements	131
Heading Sizes and Type Treatments	132
Normalizing Heading Dimensions	132
Heading Accents	133
Link Markup	133
Link Attributes	134
Virtuous Use of the href Attribute	134
Linking to Specific Passages Within Documents	135
Creating Effective Link Content and title Values	136
Styling Links	137
Link Pseudoclasses	137
Using display: block to Increase the Footprint of a Link	138
The text-decoration Property	139
The cursor Property	140
Adding Semantic Value with Inline Elements	140
Quotations	142

9. Colors and Backgrounds	143
Usability, Accessibility, and Color	144
The Additive Color Model	144
The Subtractive Color Model	145
Design, Contrast, and Complements	146
Identifying Colors, in Brief	147
Display Environments and the Web-Safe Palette	148
Creating Your Own Palettes	149
CSS Backgrounds	150
Setting background-position Values	151
The CSS background Shorthand Property	152
Composing Background Images	152
“Faux Columns”	154
Tiled Background Textures and Patterns	155
Large Background Textures and Nonrepeating Devices	156
Drop Shadows, Gel Effects, and Rounded Corners	157
Bitmapped Copy and Fahrner Image Replacement	157
The FIR Stylesheet Rules	159
Drawbacks of FIR	159
Reducing Server Load with Sprites	160

10. (Data) Tables	163
The Disadvantages of Layout Tables	163
Source Order: Square Peg, Round Hole	163

Table of Contents | xi

CSS Zen Becomes a Myth	164
Template Slavery Is Unavoidable	164
Positioning Is Rendered Useless	164
The Parts of a Data Table	165
Example: The Full Smash of Table Markup	166
Composing Cells	168
Table and Data Composition	170
Table Headers, Footers, and Heading Cells	172
Attribute and Child Selectors	173
Reducing Header and Footer Contrast	173
Adding Rollover Accents to a Table	175

11. Images and Multimedia	177
Replaced Elements	177
Preparing Images for Production	178
The alt Attribute Explained	179
Image Dimensions and Borders	179
Image Production	180
Cropping	180
Matting: Creating a Virtual “Frame”	181
Resampling: Altering the Absolute Size of an Image	182
Level Changes: Optimizing the Contrast of Photographs	183
Applying Multiple Adjustments	185
Working with Color Profiles	185
Image Optimization	186
Choosing the Right Image Format	186
Finding the Happy Medium Between Size and Quality	187
Publishing Images	188
Keeping Images Organized	188
Image Publishing and Management in a CMS	189
Image Publication Etiquette	190
Styling Images and Plug-in Content	190
Composing Image Layout Within a Column	190
Captioning Images	191
Working with Previews (Thumbnail Images) in a Gallery or Slideshow	192
Lightbox: Previews, Galleries, and Slideshows	194
SlideShowPro	194
Adding Motion and Sound: Using SWFObject to Insert Flash Videos and Presentations	195
Inserting Unwrapped Multimedia	196

A Tale of Three Companies 197 Enter Flash 197

xii | Table of Contents

Using Bare Markup to Publish Multimedia Content 198 A Caveat of Plug-in Content Styling 198 Sidestepping Plug-ins with the HTTP Content-Disposition
Header Field 199 Keeping an Open Mind 199 The video and audio Elements (HTML5) 199 The canvas Element (HTML5) 201

12. Web Typography

203 A Brief History of Letterforms 203 Origins of Modern Western Letterforms 204 Gutenberg's Press and the Art of Typography 204 The Emergence of Digital Typesetting 205 Different Limitations Without Changed Expectations 205 A Visual Glossary of Typography 206 Aliasing and Anti-Aliasing 210 Type Styles, Readability, and Legibility 212 Styling for Readability 212 Styling for Legibility 213 "The Fold" and Tiny Type 213 Sizing Type 215 Choosing the Right Units for Sizing Type 216 Em/Percentage Size Telescoping 216 Size Keywords 217 Working with Typefaces and Fonts 217 The Challenge of Limited Choices 217 Applying Type Choices: the font-family Property 220 Finding Canonical Typeface Names 222 Accessing System Default Type with the font Property 222 Character Encoding in Brief 224 What Is Character Encoding? 224 ASCII, ISO 8859-1, Unicode, and UTF-8 225 Choosing an Encoding Scheme 225 Inserting Entities to Provide Non-ASCII Characters 226 Creating Balanced Type Treatments 228 Predictability, Preference, and Panic 228 Assessing Content Scope 229 Distinguishing Type: Face, Size, Weight, Style, Color 230 Setting Type Around Blowouts 232 Styling Passages of Similar Priority 232 Enter Type Treatments 233 Typographical Miscellany in CSS 234 The line-height Property 234

Table of Contents | xiii

The font-variant and text-transform Properties 235 The letter-spacing and word-spacing Properties 236 The white-space Property 236 The Practice of Good Web Typography 236

13. Clean and Accessible Forms

237 Building Effective Forms 237 Web Applications, User Perspective, and Design Choices 237 Organizing User Interfaces by Function 238 Ten Rules for Effective Web Forms and Applications 239 Assessment and Structure 241 Establishing Requirements 241 Markup and Structure 243 Basic Form Structure, Presentation, and Behavior 246 Form-Originated get Requests 247 The post Method and File Uploads 249 Manipulating the Size and Appearance of Individual Controls 249 Prototyping and Layout 251 Prototyping 101 251 Design Patterns, Style Resets, and Form Layout 252 Grouping Controls by Appearance 254

Required Fields and Other Submission Constraints	255
Identifying Required Fields	255
Discovering and Identifying User Input Errors	256
The disabled and readonly Attributes	257
Creating Accessible Forms	258
Implementing Forms for Accessibility	259
Supporting Keyboard Navigation of Forms	260
Form Features in HTML5	261
New Input Types	262
The required Attribute	262

14. The Bad Parts	265
The Numbing Nature of Internet Explorer (Especially IE 6)	265
Browser Wars 2.0	266
Absent or Poor Selector Support	267
hasLayout	268
Margin Doubling	268
expression() Values	269
ActiveX Filters and Transitions	269
PNG Support (or Lack Thereof)	270
Poor Property Support	270
Issues with XHTML and XML	271

xiv | Table of Contents

Systemic Ugliness	271
Template Fragility and Third-Party Content	272
Markup Validation As a Prerequisite to Proper Style Implementation	272
“Best Viewed with”	272
Graded Support	273
embed Versus object	274
Form Controls, Plug-in Instances, and Element Stacking	275
Invalid Markup for Stupid Reasons	276
HTML’s Bad Neighborhoods and Cul-de-Sacs	276
Frames	277
The strike Element	278
The name Attribute	279
The noscript and noframes Elements	279
Semantic Contortions and the Limited Vocabulary of HTML	280
Inline Presentation Elements	280
Manipulating Vertical Space: hr and br	281
The pre Element Versus the white-space Property	281
CSS Travesties	282
@-Rules	282
Computed Values and Rounding Differences	282
Vendor-Specific -moz and -webkit Property Prefixes	283
The inherit Value	283
Hiding Stuff: z-index and clip	284
Counters	284
Element Flow Rules	285
Unicode Code Position Values and the content Property	285
The Awful Parts	286
The marquee and blink Elements	286
MSIE User Interface Properties	287
The align Attribute	287
The style Attribute	287
div-itis	288
Event Handler Attributes	288
Gratuitous Underlining	289
The http-equiv Attribute	289
Picking Up the Pieces	290

Appendix: URIs, Client-Server Architecture, and HTTP

. 291 Glossary

. 297

Index

. 303

Preface

HTML and CSS are old technologies that have seen over a decade of use and continue to evolve. Web developers celebrating their fifteenth year of work have seen all kinds of projects built across a wide variety of browsers, experimented with different features, and noted their successes and failures.

Despite their best efforts, the people who created HTML and CSS didn't always get it right. Some experiments didn't work out very well. At the same time, some pieces proved even more useful than expected. Mastering these technologies requires figuring out which pieces of the specs are cruft, in urgent need of abandonment, and which are gold, deserving maximum use. Focusing on HTML and CSS best practices does more than help you create sites that work: it lets you build more effective sites more efficiently, with much lighter long-term maintenance costs.

The Who and What of This Book

Hopefully you're holding this book because you read a glowing review on one of your favorite websites, or because somebody you know said that you *absolutely need* to read it. (An author can dream.)

Still, you need more information than that. Is this book for you?

If you and your priorities are described in the paragraphs that follow, then you should walk out of the store with this book under your arm, or at least sit down in the nearest available chair and start reading.

What Are the Good Parts?

There's no getting around the fact that long stretches of HTML and CSS are boring. I mean *sleep-through-it* boring. In this way, web technologies are like a certain class of movies: viewers find themselves wanting to skip the exposition so they can watch the good parts.

This book attempts to cater to that sentiment. All of the exposition—which I *do* invite the reader to tackle—is tucked away into Chapters [2](#) and [3](#), available for a quick “re wind” if you realize that you might have missed something.

The nonexpository parts are about making cool stuff happen: nailing down faithfulness to composites, getting the upper hand over bugs, building template markup that can survive redesigns, and manifold other topics.

What You Should Know Before You Read This Book

This book makes one basic assumption: that you’re familiar with the scope of HTML 4.01 elements, CSS selectors, and CSS property/value pairs. The [companion website](#) for this book includes reference tables that link to exhaustive descriptions of HTML and CSS on third-party sites, but it will be far easier to follow along if you’re already familiar with the capabilities of HTML and CSS.

In addition, this book will be easier to digest if you’ve gained an understanding of the separation of behavior, presentation, content, and structure into separate layers within a site or application.

If you feel uneasy about any of this, O’Reilly’s *Definitive Guides* and *Pocket References* for HTML and CSS come highly recommended.

For the benefit of readers who may have overestimated their knowledge, the basics of page, stylesheet, and element structure *are* covered as briefly as possible.

The Ideal Reader

You might be an ideal reader of this book if:

- You’re confident when the time comes to start building the server side of an application, but redesigns get on your nerves because you’re forced to dive back into the code and revise the bits of markup that are interspersed within it. The most effective solution to this problem is called the “CSS Zen” technique, exemplified by Dave Shea’s [CSS Zen Garden](#). This book explains CSS Zen—structuring production of markup so that redesign efforts can be confined to stylesheets—from a perspective suited to engineers.
- You’re skilled at the use of a web-centric Integrated Development Environment (IDE) such as Adobe Dreamweaver or Microsoft Visual Studio, but your expectations routinely collide with its limitations. Left unattended, an IDE typically inserts all manner of cruft (i.e., “excess; superfluous junk”) into web materials, egregiously violating the KISS (Keep It Simple, Stupid)

Principle. This occurs because IDEs are one-size-fits-all solutions. This book explains HTML and CSS in enough detail that you can start configuring your tools of choice to handle the specific cases you work with every day.

xviii | Preface

- You have—for whatever reason—a lot of bad habits that need to be superseded by good ones. Some of you probably still use HTML to manage presentation as well as structure, and CSS meanwhile is terse to the point of impenetrability. This book’s perspective places CSS in a useful light.
- You’re a print-trained graphic designer who needs to understand the strengths and limitations of the web medium in order to avoid career stagnation. You’ve looked at HTML, you’ve looked at CSS, and you believe they fit together—but you just don’t understand *how*. This book takes a close look at the connection between the two, so that you can get the hang of putting design elements exactly where you want them.
- Your professional role encourages or perhaps even requires you to develop statutory accessibility requirements, or internally mandated cross-media usability requirements. Without CSS-ready markup, there’s little hope of developing cross media-friendly sites, much less sites accessible to impaired users. This book explains how to develop a site so that accessibility requirements can be met without needing to build multiple sites in parallel.
- You’re already a specialist in some skill set outside of the presentation layer, and you want to make your job easier. Put simply, narrower specialization leads to reduced skill overlap, which in turn poses barriers to intrateam communication. This book lays out the priorities of the developers whose work lies closest to site visitors, and in so doing gives you the information you need to communicate more effectively with your teammates.
- You’re tired of beating your head against the brick wall more commonly known as Internet Explorer 6. Several sites, particularly [Position Is Everything](#), delve into solutions for the nightmare that is stylesheet authoring for legacy versions of Internet Explorer. However, most online resources are tuned to specific bugs and behaviors. In [Chapter 14](#), you’ll find condensed explanations of the quirks “under the hood” that cause unwanted collisions and blowouts, as well as a cookbook of practices and techniques that will help you avoid many such problems altogether.

A Warning About Familiarity (or Lack Thereof)

Chances are that you are already familiar with some of the contents of this book. Because its audience comprises a wide range of specialists, there may be times when material meant for engineers is painfully obvious to designers, and vice versa. There may also be times when the discussion begins to remind you of a contentious argument. Creative and implementation decisions are too

often made from a position of political strength instead of merit, and it's my hope that this book can be used to support merit based arguments against Bad Ideas.

If instead everything in this book is new, it's possible that you've gotten a bit ahead of yourself. The [book's companion website](#) is built in large part to meet the needs of folks like you, by way of ensuring that *all* purchasers of this book will be able to get some

Preface | xix

value from it. However, if the material does seem a bit advanced, you can expect some difficulty. The best way of dealing with *that* is to be patient, and ask lots of questions of colleagues and associates.

Objectives of This Book

This book is meant to translate into plain English the quirks of HTML, CSS, and the document tree that are hard to grasp without guidance or experience:

- Choosing and using the ideal version of HTML for your project
- Removing the obstacles between your current practice and consistently valid markup
- Using HTML to implement for structure, rather than presentation, in ways that get the best out of CSS
- Obscure-yet-useful HTML elements
- Getting-plug-in-content-to-work-dammit
- Using tables properly, and getting the most out of them
- The method behind the madness of CSS selectors, particularly descendant selectors
- CSS selector precedence
- The CSS block layout context
- CSS margin collapsing
- Bugs and other oddities imposed by Internet Explorer 6
- Wrangling form presentation
- The history behind the bugginess of web browsers
- What HTTP does when your back is turned (and why it's important)

This book tries to cover what all presentation layer developers *should* know. It aims to describe the many *relationships* between layers of the web technology stack that are touched by designers and presentation layer developers, and also to present the strengths of HTML and CSS.

This book will also introduce the less experienced reader to a long list of CSS layout “tricks” essential to the demands of presentation, accessibility, and Search Engine Optimization (SEO). These include:

- Centering content

- Using enhanced Fahrner Image Replacement to implement bitmapped heading type
- Creating well-aligned columns of equal (or apparently equal) height
- Using the CSS float property to get the best of both column presentation and markup source order

xx | Preface

- Building versatile, visually rich navigation interfaces
- Developing work habits that will make your sites Ajax-ready
- Getting the most out of the CSS position property
- Creating versatile grid systems for your sites

A full reading of this book should imbue the reader with the majority of the knowledge needed to transform nearly any consistent set of composites—no matter how far-out their apparent requirements—into the presentation and content layers of an accessible, usable, and “crawl”-able website.

What Is Not In This Book

This book focuses tightly on practices that maximize the effectiveness of markup and stylesheets. For that reason, a number of things are not included in this book:

Sparsely supported bits of advanced and platform-specific CSS

You can do a lot of fun stuff with CSS...but unfortunately, some of it relies on unevenly supported CSS selectors and properties. Such cases will be handled in terms of *desired results*: if an ActiveX filter supported in Internet Explorer has an analog in Firefox, it might be mentioned, or vice versa for -moz-* properties that have analogs in the IE runtime environment. The *minimum* requirement for discussion of implementation techniques in this book is reliable support in both Firefox 3 and Internet Explorer 8, and broader platform support for techniques that render obscure accents.

CSS properties targeted at comparatively obscure media types

This book will cover production techniques well suited to the creation of highly accessible sites, but it is only intended as an *introduction* to implementing sites that are accessible to impaired visitors.

JavaScript and the Document Object Model (DOM)

While this book will mention JavaScript at times and even occasionally show a bit of code, its focus on HTML and CSS means that it doesn't cover how to manipulate HTML and CSS with JavaScript or the DOM.

Integration with frameworks such as jQuery and YUI

Many people have many beautiful things to say about JavaScript frameworks, but you won't find any mention of them in this book. Despite their usefulness in a variety of environments, JavaScript frameworks are neglected here for reasons of scope. The best resources for learning about

the interaction of JavaScript frame works, styles, and markup are to be found in web resources and books that focus on frameworks specifically.

Comprehensive discussion of CSS frameworks such as YUI Grids and Blueprint The goal of this book is to help you burnish your skills in good faith so that the results on your résumé are pleasing not only to Human Resources evaluators, but to hiring managers as well. Therefore, reading this book should help you to better

Preface | xxi

understand any CSS framework that you might be called upon to use, instead of instructing you on the use of any framework in particular.

Web server configuration techniques

Typical web server runtime configurations neglect a number of settings that can ease the achievement of usability, accessibility, and standards compliance objectives. However, these oversights fall more into the domain of system administrators. A number of other O'Reilly titles, particularly *Webmaster in a Nutshell* and *Website Optimization*, address this area of interest. A number of online communities and blogs also explore this topic from time to time.

Developing for the mobile web

This book has the misfortune of being written by a lifetime resident of the U.S., where the feature set and reliability of mobile web access has plenty of room for improvement. The iPhone's popularity has improved the situation, but still has not made it entirely tolerable. As it stands, only a minority of the mobile device users in the U.S. can hold any realistic expectation of using the same Web as personal computer users. Meanwhile, the expense of prepaid device connectivity found in the U.S., and the wildly uneven availability of unencumbered emulators for mobile device platforms, further exacerbates the problems faced when developing mobile content for U.S. visitors. It is my hope that the *next* edition of this book will be able to include development techniques intended to benefit site visitors who use mobile devices.

Any mention of the Opera desktop browser

If there is one omission from this book over which I agonize, it's the omission of the Opera desktop browser from all discussions of browser behavior. Unfortunately, when I weighed Opera's market share against the amount of testing its inclusion in the book would require, the results of the comparison were superlatively discouraging. Since I owe Chris Mills of Opera *direct* thanks for his role in helping me to secure the contract for this book, rest assured that I did not make my decision lightly. Given any more than the barest amount of reader interest, I won't hesitate to discuss the Opera desktop browser at length on this [book's companion website](#).

About Web Standards

Last but not least, there is the question of compliance with World Wide Web

Consortium (W3C) Recommendations in commercial settings, particularly those environments that are nurtured in large enterprises.

I've always made it a point to distinguish between "standards friendliness" and "standards compliance." The first obeys the *spirit* of so-called web standards and is easy to achieve with practice, while the second focuses on obeying the *letter* of the Recommendations and can prove impossible to achieve.

xxii | Preface

The effectiveness of a website is enhanced far more by standards friendliness than by standards compliance, with the greatest enhancements coming from adherence to both objectives. This book embraces the compromises and fallbacks that preserve standards friendliness in spite of adverse development conditions, with only the occasional twisted grimace.

You may have noticed that I referred to "so-called" web standards earlier. The underlying irony is that web standards...aren't, at least not literally.

Standardization requires conscientious use of a formally defined system across an entire industry, typically (if not always) by standards bodies whose work contributes directly or indirectly to policies and publications of the International Organization for Standardization (ISO).

Another hallmark of true standards is an objective set of criteria and processes by which claims of compliance can be *enforced*—an asset that the W3C's products very much lack.

For these reasons the popular definition of W3C Recommendations as *standards* is reasonable in spirit, but has no basis in literal fact.

That said, the practice of web standards development has evolved tremendously since the go-go era of the 1990s, a point that's explored in greater detail on [this book's companion website](#).

About Photoshop

Chapters 9 and 11 discuss image production techniques in some detail, and the procedures described there are based on the Adobe Photoshop user interface. I took this approach because in any moderately sized group of web professionals, you'll find a wide diversity of preferred tools and implementation techniques...*until* you get to the question of working with graphics. Alternatives to Photoshop (particularly Fireworks, another Adobe product) claim their devotees, but even those operators will agree that a working knowledge of Photoshop's toolset and user interface is immensely useful.

My choice was also based on slanted experience; I haven't used anything other than Photoshop to manipulate web images since I was a full-on novice. My hope is that visitors to [this book's companion](#) site will submit their own alternative-title cookbooks for the image manipulation techniques discussed in the book.

The matter of relying on Photoshop also illuminates the importance of tool choice with respect to team effectiveness. [Chapter 4](#) introduces the value of production standards and code libraries, but the benefits of tool uniformity also extend to off-the-shelf software choices.

What You'll Find on the Companion Website

The companion website to this book, www.htmlcssgoodparts.net, contains a wealth of information. Among the goodies you'll find are:

- Errata and corrections
 - Blog entries about reader questions, current technical developments, and best practices
 - Staged demonstrations of techniques discussed in the book, complete with source markup and stylesheet rules and indexed to page numbers
 - Boilerplate and/or templates for multicolumn layouts and other widgets
 - HTML and CSS reference tables that link to multiple third-party documentation sources
- Visitor-submitted reviews of books and software of interest to this book's audience

Nomenclature

Names for the various pieces of web technology sometimes vary from shop to shop and from place to place. To minimize the potential for confusion, the terms spelled out below in *emphasis* are used consistently throughout the book.

Files are discrete nodes on a server host's native filesystem, while *resources* are documents or document fragments referenced by discrete Uniform Resource Identifier (URIs). Not all files are URIs, and not all URIs are files; a URI might contain several files, database query results, or data streams, while a file might amount to nothing more than the logic that determines the content of multiple URIs.

Pages or *documents* contain one or more resources of arbitrary classification and are the visitor-facing output of a request for a single URI (or perhaps multiple URIs, on sites where Ajax has been deployed). Finally, this book treats the differences between the terms "URI" and "URL" as minor to the point of insignificance, in part because the term "resource" itself has been so muddled it's become functionally meaningless in the face of rapid evolution.

Content is the matter around which websites are built.

HTML, XHTML, and XML tags are referred to in sum as *markup*.

Stylesheets are the content of CSS files or style elements. Stylesheet *rules* assign presentation to one or several elements within a page. A stylesheet rule contains a *selector*, which defines the element(s) on the page to which one or more *property/value pairs* are to be applied.

Browsers are also known as *user agents*, *UAs*, or *clients*.

HTML and CSS are *parsed* in serial fashion, and according to the results of that process the browser *renders* a page.

xxiv | Preface

JavaScript is a registered trademark of Sun Microsystems that refers here to the programming language used to script data processing and interactivity within browsers. Different vendors refer to it by different names to avoid court trouble, but where there's a browser, there's usually a JavaScript interpreter.

The *Document Object Model* (or *DOM*) is both the representation of a web document's structure, and the definition of how that structure ought to be organized, queried, and altered programmatically. Several DOM specifications for web documents exist, though only one is developed and sanctioned by the World Wide Web Consortium as a body.

The stack of web-related services is colloquially and commonly understood to include an operating system, a web service, a relational database service, a server-side scripting language, HTML, CSS, and JavaScript. The platforms used in the first four layers of the stack vary from shop to shop. Of the layers on this notional stack, the first four layers refer to the *server-side* environment, and the latter three to the *client-side* environment.

The client-side environment is artificially divided into four sublayers: *structure* (defined by markup), *content* (enclosed by markup), *presentation* (defined by CSS), and *behavior* (defined by JavaScript). Together these form a second Model-View-Controller (MVC) architecture that mirrors and interacts with the MVC architecture on the server side.

Ajax is an acronym representing *Asynchronous JavaScript And XML*, an implementation approach made convenient by the ubiquity of the XMLHttpRequest Application Programming Interface (API).

HTML *elements* are the principal items in the HTML namespace; *tags* are literal markup, which might well contain *attributes* with *values*, and most often enclose content.

Copy and *illustrations* are to content what *text* and *images* are to data.

A *doctype declaration* can (and usually should) appear at the beginning of a given web document and identifies the version of HTML against which that document should validate. The *document type definition* (also called a *DTD*) is a machine-readable series of statements that defines validity for the applicable version of HTML. The values contained in a doctype declaration directly reference a specific DTD.

W3C Recommendations are official documents that serve as specifications for

web technology platforms and best practices associated with the use of those platforms.

Preface | xxv

Project managers minimize the obstacles standing between a project team and the completion of their deliverables. *Designers* create the look, feel, and user experience of sites. *Engineers* and *application developers* design and write the code that makes sites go. *Presentation layer developers* as a group deliver everything that directly faces site visitors; of these, *stylists* create templates and stylesheets, and *producers* ensure that content gets placed into production. Most other roles commonly found in web project teams are titled here as they would be in an advertising/marketing environment.

Current browsers or user agents refer to the mass-market browser versions current when this book went to press: Internet Explorer 6–8, Firefox 3.x, and Safari 3.x–4.x.

Several of the terms listed here point to obscure processes with an impact on the web user experience; these processes will be discussed in more detail throughout this book.

“Read the Source, Luke!”

When I first started working with the web platform in 1995, “Read the Source, Luke!” was easily the most popular advice given to the greenest newbies on mailing lists. This hearkens back to the climactic moments of *Star Wars: A New Hope*, and exhorts the petitioner to read through the source markup (and now, 13 years later, the stylesheet rules) of results they find admirable.

There’s more to this advice than sci-fi nerd humor. The best understanding of effective passages of markup and styles comes from reading through them without filters—in much the same way that “Force-sensitives” of the *Star Wars* milieu get the most out of *their* talents by letting go of their prejudicial thoughts.

If you try to puzzle out how somebody accomplished a presentation goal *before* you read his source, you might be badly disappointed...and if you *never* read his source, you might never figure it out for yourself.

However, before we can get into the finer points of learning from source markup and CSS, it’s best to look at the Web as a system—the relationships between the underlying conventions and technologies that make it go.

Conventions Used in This Book

The following font conventions are used in this book:

Italic

Indicates pathnames, filenames, and program names; also Internet addresses, such as domain names and URLs

Constant width

Indicates command lines and options that should be typed verbatim; names and keywords in programs, including method names, variable names, and class names; and HTML element tags

xxvi | Preface

Constant width bold

Indicates emphasis in program code lines

Constant width italic

Indicates text that should be replaced with user-supplied values

This icon signifies a tip, suggestion, or general note.

This icon indicates a warning or caution.

Using Code Examples

This book is here to help you get your job done. In general, you may use the code in this book in your programs and documentation. You do not need to contact us for permission unless you're reproducing a significant portion of the code. For example, writing a program that uses several chunks of code from this book does not require permission. Selling or distributing a CD-ROM of examples from O'Reilly books does require permission. Answering a question by citing this book and quoting example code does not require permission. Incorporating a significant amount of example code from this book into your product's documentation does require permission.

We appreciate, but do not require, attribution. An attribution usually includes the title, author, publisher, and ISBN. For example: "*HTML & CSS: The Good Parts*, by Ben Henick. Copyright 2010 Ben Henick, 978-059615760-9."

If you feel your use of code examples falls outside fair use or the permission given above, feel free to contact us at permissions@oreilly.com.

Safari® Books Online

Safari Books Online is an on-demand digital library that lets you

easily search over 7,500 technology and creative reference books and videos to find the answers you need quickly.

With a subscription, you can read any page and watch any video from our library online. Read books on your cell phone and mobile devices. Access new titles before they are available for print, and get exclusive access to manuscripts in development and post feedback for the authors. Copy and paste code samples, organize your favorites, download chapters, bookmark key sections, create notes, print out pages, and benefit from tons of other time-saving features.

Preface | xxvii

O'Reilly Media has uploaded this book to the Safari Books Online service. To have full digital access to this book and others on similar topics from O'Reilly and other publishers, sign up for free at <http://my.safaribooksonline.com>.

How to Contact O'Reilly

We have tested and verified the information in this book to the best of our ability, but you may find that features have changed (or even that we have made a few mistakes!). Please let us know about any errors you find, as well as your suggestions for future editions, by writing to:

O'Reilly Media, Inc.
1005 Gravenstein Highway North
Sebastopol, CA 95472
800-998-9938 (in the U.S. or Canada)
707-829-0515 (international/local)
707-829-0104 (fax)

O'Reilly's catalog page for this book, which lists errata, examples, and any additional information, is at:

<http://www.oreilly.com/catalog/9780596157609/>

The author has a companion website for this book at:

<http://www.htmlcssgoodparts.net>

To comment or ask technical questions about this book, send email to the following, quoting the book's ISBN (9780596157609):

bookquestions@oreilly.com

For more information about our books, conferences, Resource Centers, and the O'Reilly Network, see our website at:

<http://www.oreilly.com>

Acknowledgments

When I reflect upon my experiences of 15 years as a site builder, the quality that impresses me most is ignorance. There's plenty of it to go around, and like many site builders, I often take opportunities to castigate the ignorance of others less skilled...but not in this book.

Why?

Of greater concern still is my own ignorance, which is no less deserving of criticism. Close on the heels of ignorance are trepidation and obstinacy, both of which were

xxviii | Preface

regular contributors to my internal dialogue during the year that I took to write this book.

Given that attitude, this book attempts to exemplify the belief that one should light a candle for others to find their way, instead of cursing the darkness. I give fair due to the comfort engendered by continued reliance upon legacy production techniques, and where best practices are mentioned, I make a point of selling them as softly as I can without muddling my message.

In sum, I tried to fill this book with the advice that would have stood me in good stead eight or nine years ago, that instead many people (including myself) sorted out only by trial, error, and accident, and thence shared one iota at a time as they became able.

I hope that this book will be as useful to you now, as it would've been to me when I was working toward CSS mastery.

There are a number of people whose involvement in my life brought me far enough to achieve that state of mastery and to write this book. Since this is my first chance to call them out fully in public, I feel that I ought to mention them by name. Apart from my family, these benefactors include Christian Cepel, Steven Champeon, Sumin Chou, Teddi Deppner, Nick Finck, David Hemphill, Molly Holzschlag, Brenda Houston, Ethan Marcotte, Doug Petersen, Lance Taylor, Thomas Vander Wal, Peter Zale, and Jeffrey Zeldman. These individuals have each made significant contributions to my life, and without *all* of them, it's likely that this book would never have been written.

There are also several people named in the book itself. Of these, Chris Mills of Opera Software has my special thanks. Chris has never been far from this project—he's the one who suggested me to O'Reilly Media as an author candidate. In fact, Chris started me down this road in the first place, by inviting me to contribute to the [Opera Web Standards Curriculum](#).

The contents and quality of this book are not owed to my work alone. In fact, it was kept from the precipice of failure by the indefatigable patience of Simon St.Laurent, my editor at O'Reilly Media. My words might be on these pages and my name might be on the cover, but Simon's constant support of this project bridged the long gap between my effort and a successful conclusion.

Michael Smith is ultimately responsible for this book's contents on the subject of HTML5, and the absence of his name from its cover makes poor thanks for his will ingness to rescue me from lurching through that proverbial minefield.

I had the opportunity to handpick three technical reviewers: Kimberly Blessing, Gez Lemon, and Chris Van Domelen. Each of them made categorically critical contributions to the accuracy and currency of this book, any remaining lack of which is my respon sibility alone.

Kimberly and Chris have also been stalwart associates and sources of technical advice for several years, and I find myself unable (as in so many other cases) to thank them in adequate measure for their help.

Preface | xxix

O'Reilly Media was gracious enough to provide three additional technical reviewers: Edd Dumbill, Elaine Nelson, and Shelley Powers. Their contributions helped find many more glitches and improve the structure of the book.

While I might've written this book someday, you wouldn't be reading it now without the outstanding work of Douglas Crockford, which proved that a "Good Parts" series would find enthusiastic readers.

I believe strongly that things *really* work on account of the work done by "backstage" folks, and this project confirmed that belief. Especially high praise goes to Emily Quill, who untangled the unwieldy parts of this book's draft, and in doing so, ensured that you will get your money's worth for this book. Loranah Dimant tirelessly addressed my last-minute edits and ensured a bright polish for the book.

My final thanks here go to Eric Meyer, who sets the bar high for the rest of us who take a hand at developer education.

In closing, I hope that the knowledge you gain from this book will lead you to achievements that are no less impressive in degree than those of the people mentioned here.

CHAPTER 1 Hypertext at the Core

A properly built website is far more than the sum of its markup, stylesheets, scripting, and multimedia resources. Well-built websites take full advantage of their hypertext medium, making a once obscure technology central to the way we consume information. Without easily activated *links*, the Web wouldn't be the Web; it would be just a rigidly organized heap of documents.

While hypertext offers tremendous flexibility, it also requires developers to help visitors find their way. Visitors will take unexpected paths even within a site, and will arrive from sites or bookmarks that you don't control. The power that hypertext provides also comes with the responsibility to structure your site in ways that visitors will be able to comprehend and navigate.

The Web Without Links

The Web's use of links to connect information makes it different from previous media. Today, when the Web is so familiar, it's easy to forget those differences, but they pave the way to developing successful websites. So what happens when you remove hyper links from a site?

- The first and most significant result of excising hypertext from a networked information system is that *content becomes strictly linear*: one must first read through a given amount of content before reaching the object of his interest. Take the links out of hypermedia and the result is nearly useless without a concerted attempt at imposing internal order and structure.
- Linear resources are *designed and structured on different assumptions*, expecting that a reader has examined (or at least referred to) previous passages of content. Take this book as an example. You can jump around within it, but chapters are still ordered by the descending importance of the subjects that they cover. Also, if the [companion website for this book](#) did not exist, there would be plenty of verbose markup examples between its covers.

1

- *The visitor's sense of location is informed by standard cues.* Most books and other linear information resources have some sort of header or footer content on every page (or on the title bar of the reader application), and a visitor's state of progress through a networked information resource, like a large Portable Document Format (PDF) file, is cued by a vertical scroll bar.

These distinctions illustrate how *hyperlinks* add new dimensions to documents. While this gives the Web tremendous flexibility, it also creates challenges. The added navigational possibilities result in systems that make it difficult to maintain a sense of place. While the consumer of a linear resource can count on traditional cues and her own critical thinking skills to enforce her sense of place, the consumer of hyperlinked resources needs the help of designers and implementers to maintain her sense of location.

Notions of "beginning" and "end" are artificial if not entirely absent from web media. This is a marked departure from the fundamental nature of nonhyperlinked resources, which are bounded by definition.

URIs

In a perfect system, Uniform Resource Identifiers, or URIs (formerly Uniform Resource Locators, or URLs) would be hidden from the site visitor. They aren't especially human-readable, comprised as they are of a protocol token, a host alias, and something that looks like a filesystem reference but isn't. URIs often end in token/value pairs that are deliberately designed to be computer-readable, as opposed to visitor-friendly.

We're all familiar with simple URIs like `http://www.example.com/` that point to the home page of a site. These appear in advertisements and on business cards, and the `http://` has come to mean "type this in to find the website." However, well-crafted URIs can contain a lot of information—look at commonly encountered URIs at your favorite search site or news site, and you'll see a lot more going on. Google search result URIs, for example, can contain a parameter named `start` that specifies the number of results ranked higher than those displayed, as in `http://www.google.com/?q=hyper text&hl=en&start=10`. In a similar vein, popular Content Management System (CMS) platforms and e-commerce catalog platforms allow the same resource to be associated with *multiple* URIs, where the longer URIs enhance a resource's searchability or specify that additional content be served along with the core resource (e.g., a product listing or the summary of a weblog entry).

Browsers and other tools use the HTTP protocol to process URIs and retrieve information. If you want to know more about how this processing works, and how its features and limitations might affect your pages, see the [appendix](#).

2 | Chapter 1: Hypertext at the Core

Managing Links

Hypertext as we understand it today was first implemented at Stanford University in the 1960s, but didn't become an everyday tool until the advent of affordable commercial Internet access three decades later. The "explosion" of the Internet not only provided a way for hyperlinks to connect across a broad network, but it also nurtured an understanding that web hyperlinks should be simple and tolerant of failure.

HTML link conventions assume that the person creating a link knows what will be at the URI at the end of it. That doesn't necessarily mean that link creators *control* what is at the end of the link, however. In fact, the ability to link to any content without having to ask its creator beforehand is a critical aspect of the Web's success. If it has a URI, you can link to it. If a URI doesn't work, a well-built site will report an error (like the ubiquitous "404 Not Found") and present a page that can help the lost visitors find their way again.

The power and immediacy of web links raised all kinds of cultural (and in some cases legal) questions about what it means to be able to link directly to

someone else's material, but over time a simpler and probably more intractable issue arose: *link rot*. Creating links to information you don't control eventually means that over time those links break, as information and even sites change or disappear. It also means that you may have visitors arriving at your site who are confused and frustrated because they didn't find what they wanted immediately.

To some degree, link rot is inevitable, and even automated systems (like search engines) have a difficult time keeping up with it. Even if links still point to useful pages, they may evolve over time into something very different. Within your own sites, you have somewhat more control, though major site redesigns can make this difficult. Caution, well-built error pages, and clear navigation can help minimize these problems.

While visitors can usually deal with regular links that send them to the wrong place, it may be more difficult for your pages to recover from missing images, code, stylesheets, or other components that are supposed to be inserted via accurate href and src values. The more important the component to your page, the more you will want to link to it at a stable location under your control.

Improving the User Experience with Linking

Links are part of HTML, the means by which URIs are most commonly exposed within the Web's application layer, at the point where HTML and HTTP intersect. At the application level, there isn't much difference between following a link and accessing a given URI through the Location bar of a browser.

URIs | 3

Links provide infinite opportunities to site builders—opportunities that are usually passed over. Anything can link to anything else. Hyperlinks in documents aren't constrained to site navigation, stylesheet references, and syndication references; they can also point to an unlimited number of related documents and all kinds of alternative content. Hyperlinks that respond to user interaction can be placed anywhere, point to anything, and trigger behavior limited only by platform constraints, good sense, and a site builder's imagination. Well-implemented hypertext enhances information with the following benefits, among many:

Broadened accessibility to and control of information

Hyperlinks can always reference every part of the Web that is not access-controlled. Rather than delivering long chunks of exposition out of necessity (as this book does) or referring to other matters that must then be physically obtained, hyper links allow the users to decide for themselves which information resources they will access and how.

Creation of multiple narratives from a single body of content

Hyperlinks make it possible for a visitor's "journey" to take any and all forms that he desires...within reason.

Community-driven attention flow

Incoming hyperlinks lend credibility to destination content without the need for subject matter—expert intervention—a fact that defines a number of systems already in use, especially Google's PageRank algorithm. It remains possible for the "wisdom of crowds" to be qualitatively poor, but accuracy tends to increase over time since subject matter experts remain closely involved with the process.

Hypertext Implementation Challenges

Web technology allows users to direct their own experience in ways that until 1992 had been the stuff of science fiction. No single person or entity has unqualified control over a given user's web experience (although not for lack of trying). A single user session can result in requests for content from multiple unaffiliated authors, on tangential or unrelated subjects, and require an arbitrary amount of user interaction.

This seeming anarchy places new demands on implementers:

1. Context (i.e., steady "You Are Here" and "That's Over There" signaling) is the most important part of an effective site, apart from the actual site content.
2. Untested assumptions about a visitor's goals and knowledge create a short, straight path to folly and disaster.

4 | Chapter 1: Hypertext at the Core

3. Duplication of content adds needless burdens to the user experience (and to the site building process).
4. The Web's lack of bounds, assumptions, and context can create user impairments out of thin air, and often these impairments *must* be addressed. The Web's tremendous openness creates the need for specialist disciplines in web information architecture and usability.

Because the Web *breaks* the linear structure of traditional media outright, implementers must never forget that their tools define *context*, first and foremost.

CHAPTER 2

Working with HTML Markup

When building a site, one of the most important tasks that you perform is link creation, but HyperText Markup Language (HTML) offers a heap of features beyond links. HTML documents describe the hypertext and contain much of the content users explore while visiting the Web, connecting them to other resources including presentation style, scripts, images, video, sound, and much more. As you'll see, a key part of working with HTML is knowing when to let other technologies (and sometimes people) do their work.

HTML has been in constant development since its invention in 1992, and web software (like browsers and web-focused IDEs) have evolved apace. As HTML nears its third decade, clear best practices for markup have emerged both from HTML markup itself and from the technical and business ecosystems that interact with it. Clear HTML syntax lets you build a reliable document tree to hold your content and support additional layers of style and behavior. [Chapter 5](#) is devoted to the features of CSS that interact with the document tree.

HTML Syntax

HTML and its stricter sibling XHTML define a set of rules for marking up documents, as well as rules for how that markup should be structured. HTML parsers (but not XHTML parsers) usually follow a principle referred to as “Postel’s Law,” stated as follows:

Be conservative in what you send, and liberal in what you accept.

Where XHTML requires the creator of the document to write very precise markup, HTML parsers will liberally repair omissions and remove empty elements that are present in markup. This makes the document valid from the visitor’s perspective, though not necessarily using the structure originally intended by the stylist. (HTML5 is defining this behavior formally, but in the past it has varied from browser to browser.)

Tags, Elements, and Attributes

HTML defines a number of *elements*, each of which falls within a particular semantic domain and takes a name derived or borrowed whole from English. Elements define the structure of the document and lay the foundation for its

presentation and manipulation.

Each element reference in a document is contained within one or two *tags*—tokens enclosed by angle brackets (< and >) containing the name of the element being used. *Opening tags* always begin with < immediately followed by the element name, and reference all attributes and values associated with the element; *closing tags* are feature less apart from the element name, which is preceded by a forward slash.

Elements without discrete closing tags are handled in different ways:

- HTML elements with *optional* closing tags—most notably li (list item) and p (paragraph) elements—allow for complete omission of the closing tag.
- HTML elements that *forbid* a closing tag are indistinguishable from opening tags of other elements.
- XHTML elements always either forbid or require closing tags; “optional” doesn’t get much traction in XHTML.
- Tags referencing XHTML elements that forbid closing tags terminate not with >, but with />. This token is usually preceded by a space, to prevent parsing failures at the hands of legacy user agents.

Tags can contain an arbitrary amount of whitespace, and attributes can be listed in any order within an opening tag.

All element instances can be modified through the use of *attributes*, most of which should in turn be followed by *values*. In plain HTML, elements and attributes are case insensitive, but in XHTML, they should be written entirely in lowercase characters. As a dialect of XML, XHTML poses two additional rules:

- XHTML is *broadly* case sensitive, which can matter with respect to values, while HTML enforces the case sensitivity of values only for the class and id attributes.
- Where an attribute is applied within an XHTML tag, it *must* be followed with a value. In the case of attributes that are typically deprived of a value in HTML, the common practice is to duplicate the name of the attribute in the value (e.g., checked="checked" instead of simply checked).

[Example 2-1](#) shows some valid XHTML 1.0 Transitional markup.

Example 2-1. XHTML 1.0 snippet

```
<div id="header"><h1><a href="/">AcmeStore.com</a></h1></div>
```

```

```

8 | Chapter 2: Working with HTML Markup

The first line of [Example 2-1](#) contains three elements, one inside another, not unlike a matryoshka doll. It’s important to remember that when elements are nested, they should be closed in the *reverse* order in which they were opened, to create the nesting shown in [Figure 2-1](#). Inadvertent failure to follow this rule is a common cause of blowouts.

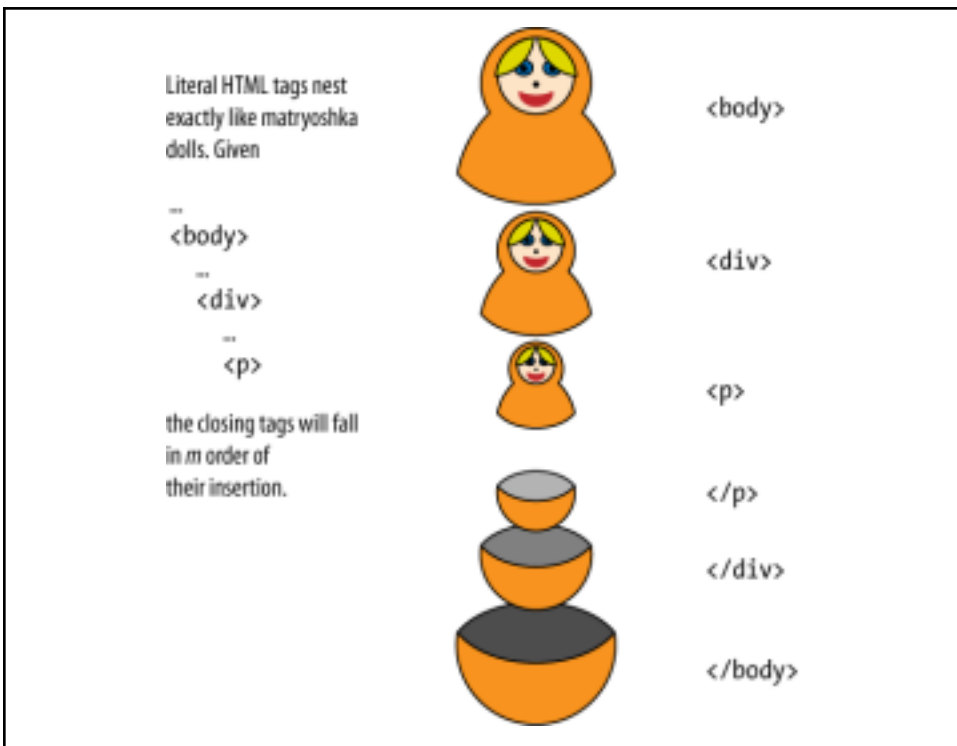


Figure 2-1. Well-formed HTML tags nest in exactly the same way as matryoshka dolls

Example 2-1 also treats attribute values according to XHTML rules. XHTML values are *always* quoted. HTML values follow a different rule:

In certain cases, authors may specify the value of an attribute without any quotation marks. The attribute value may only contain letters (a–z and A–Z), digits (0–9), hyphens (ASCII decimal 45), periods (ASCII decimal 46), underscores (ASCII decimal 95), and colons (ASCII decimal 58). *We recommend using quotation marks even when it is possible to eliminate them* [emphasis].

—HTML 4.01 specification, World Wide Web Consortium

Page Structure

When a browser receives content it believes to be HTML, it will attempt to process the content based on what it can figure out from the markup contained in the document. Even if that markup has missing parts, is structured strangely, or is otherwise not standards-compliant, the browser can usually display something resembling what its creator had in mind.

If a web document is to be *valid*, however, it *must* contain a number of properly structured elements with appropriate content. A valid HTML document contains the following components, in order:

1. The document type declaration
2. The document's html element
3. Within the html element, the document's head element
4. Within the head element, a title element and any necessary link, script, base, and meta elements
5. Within the html element and after the head element, the document's body element, which represents everything on the page that might be directly user-facing
6. Within the document's body element, at least one block element

Rendering Modes, Flavors of HTML, and Document Type Declarations

As of this writing, HTML has been steadily evolving for 17 years. Five versions have been developed, and HTML5, the most recent of these, is steadily making its way to popular use though it is not yet complete. The World Wide Web Consortium (W3C) has also published a Recommendation for XHTML, the XML-conformant version of HTML 4.01.

While it is still too early to know what the “good parts” of HTML5 will be, throughout this book we will cover new HTML5 functionality where it might change best practices.

Since version 1.0, HTML has included something called the *document type declaration* at the very beginning of the document. This identifies the version of HTML used

10 | Chapter 2: Working with HTML Markup

in a document to a user agent, but was generally ignored by web browsers until 2001. For example, the document type declaration for HTML 4.01 Strict would look like the following:

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN"
"http://www.w3.org/TR/html4/strict.dtd">
```

The most significant impact of the document type declaration is its influence on the way that element footprints are rendered. Different DOCTYPEs lead to different rendering modes. (They also set expectations for HTML validators.)

You've probably seen the acronym "DTD," which expands to "Document Type Definition." The DTD is the machine-readable definition that the document type declaration is intended to reference. At any rate, the declaration and definition are different matters; the former ideally points to the latter, and only the latter is referred to as a DTD. This [book's companion website](#) examines the finer points of DTDs and document type declarations in greater detail.

HTML or XHTML?

The currently popular "flavors" of HTML are variants of HTML 4.01. The widest divide lies between those variants that follow traditional HTML syntax, and those redefined to meet XML's requirements for well formedness. When served with the correct MIME type (see [Table A-1](#) in the [appendix](#)), XHTML is also parsed according to the stricter syntactical requirements of XML.

Normal HTML enforces somewhat looser rules, allowing things like the omission of closing tags, and is case insensitive. XHTML, meanwhile, requires all elements to be properly closed with a complete tag or `</>` as needed, and named entirely in lowercase characters.

XHTML suffers one substantial disadvantage—its canonical MIME type isn't supported by Internet Explorer, a problem discussed in greater detail in [Chapter 14](#).

However, carefully formatted XHTML (or HTML written under the constraints of XHTML) offers an even greater advantage that has led to its selection for markup examples in this book and its related materials. Because XHTML's required syntax is more rigid, XHTML source fragments are measurably easier to read than their HTML analogs, and the rules defining valid XHTML are less confusing.

HTML5 includes support for an XML syntax, but does not require its use.

Rendering Modes, Flavors of HTML, and Document Type Declarations | 11 Strict, Transitional, or Frameset?

As HTML has evolved, a number of elements have been *deprecated*: that is,

officially designated as obsolescent or out of scope. In addition, many elements have circumscribed scope—they must appear within certain elements or contain certain elements.

In brief, the differences between the Strict, Transitional, and Frameset subtypes of HTML can be defined in terms of permissiveness and rigidity. Strict variants have the narrowest requirements with respect to the contents or containers of certain elements, and are less relaxed about the use of deprecated elements.

The Frameset subtype, meanwhile, is meant to be used in one circumstance: for documents that define a series of frame elements. Framesets and frames will be discussed in more detail in [Chapter 14](#).

Finally, note that `iframe` elements fall within the scope of the Transitional document types, *not* the Frameset document types.

HTML5 offers only one choice for document type, which is effectively Strict plus new HTML5 features.

CSS3 allows those definitions to be made independently of the document type with the box-sizing property, which has two values: `content-box` and `border-box`. The layout behaviors associated with these values are given close attention in [Chapter 6](#).

A Tale of Two Box Models

Current web browsers use document type declarations as a “switch” that can determine the “box model” used to define the underlying measurements that will inform the page layout.

The dimensions of an element box defined by HTML can be modified with various CSS properties that control its content footprint, gutters (padding, etc.), borders, and margins as separate components. Older browsers calculate these dimensions *subtractively*: custom footprint dimensions *include* associated gutters and borders. Margins are also treated differently with respect to containing elements.

However, the CSS 2.1 specification requires that footprint calculations be handled *additively*, so that gutters, borders, and margins are rendered as an addition to any custom footprint dimensions. Of course, there is still a lot of content on the Web that was styled to accommodate the older layout approach.

By including a document type declaration in a document, a page author sets a switch that defines the layout model that will be applied to a page’s block elements by default. Some declarations cause rendering engines to behave according to the technical

standard set in the CSS 2.1 specification, while others cause rendering engines to behave according to the legacy approach (frequently referred to as “quirks mode”). Where a document type declaration is omitted, the legacy box model is applied.

In cases where the distinction might be important, source examples in this book and its companion materials will be framed in terms of the box model defined in the CSS 2.1 specification.

For a thorough list of DOCTYPEs and the modes that various browsers use to render them, see <http://hsivonen.iki.fi/doctype/>.

Choosing the Right Document Type for Your Project

In the hands of experienced developers, document type choices are a question of personal preference. I always use XHTML 1.0 Transitional for new projects and redesigns from-scratch—thanks to the consistency of XML’s rules, I find that the resulting markup is easier to comprehend during the production and quality assurance phases of a project.

However, my needs are not yours. In order to address your most important needs, you should ask the following questions before choosing the document type to be used on a particular project:

What “flavor” of HTML does the project sponsor use on its other online properties, or use as a matter of course?

Generally speaking, it’s best to stick with conventions established by materials already in production.

Does anyone hold the expectation that content will be stored in a datastore meant to be accessed by multiple systems?

In this case, XHTML—which is a dialect of XML—might well be a better choice, since portability is perhaps the greatest strength of XML.

How likely is it that your work product will be processed by transformation algorithms such as search-and-replace functions?

Strict document types will hold up better in the face of transformation algorithms.

Finally, on a somewhat different note, there is the question of how you use your tools. It’s generally best to hang onto the tools you’re already using, unless you have an obvious need to change.

Beautiful Parts: Universal Attributes

You'll be seeing a lot of the class and id attributes throughout this book. These are *universal attributes*—they can be used by *any* element in the HTML vocabulary that's valid as well as body itself.

In addition to class and id, there are four other attributes that are similarly

versatile: • title

- lang/xml:lang
- dir
- style

dir specifies which direction type should run. style will be described in [Chapter 14](#).

Providing Stylesheet Hooks with class and id

Two attributes that can be assigned to *all* elements are class and id. Several class values but only one id value can be assigned to a given element. Multiple class values are separated by spaces, e.g., class="alternate callToAction".

Valid class and id values should contain only letters, numbers, hyphens, and under scores. These values should begin only with letters and numbers. However, Internet Explorer 6 parses and applies stylesheet values that are associated with class values, id values, and property names that begin with underscores—an oversight that provides stylists with a low-pass filtering technique.

A more important question is *where* to put classes and ids. As a rule, classes should be assigned to those frequently encountered elements that share both design purpose and presentation peculiarities, but aren't used predictably. On many sites, classes are also assigned to the body elements of pages that fall within a single section of the site's architecture, such as the common "About" and "Contact" sections.

The overall structure of site templates tends to inform where and how id and class values are assigned. I typically assign the following ids to the appropriate elements of every site template that I build:

- main
- header
- primaryNav
- bodyCopy
- sidebar
- footer
- secondaryNav

14 | Chapter 2: Working with HTML Markup

If you look carefully at that list, you'll note the total absence of page- or site-specific values that refer to page coordinates (e.g., left or right columns), color, or specific sizes. A similar reliance upon context is used for class values as well; section values and hints to purpose such as error make frequent appearances, but references to absolute dimensions or colors do not. The closest thing to an exception in this regard is made for form styles, where minimally subjective class values like short, medium, and long turn up with regularity in order to avoid styling label/field pairs one by one.

Describing Content with title and lang

In addition to id and class, HTML 4.x and XHTML 1.x added two other universal attributes that are used to provide metadata about the language and general nature of the content related to the elements to which they are applied.

Of these, title is more commonly used. Its value is an arbitrary string that provides a brief description of an element's content. It can also include the title of a link's destination, a technique used extensively on Wikipedia. Finally, browsers can put aside title values and display them as document metadata. If implemented and used well, title values can be a tremendous help to visitors trying to find needle-sized bits of information in haystack-sized information stores.

The title attribute is comparable to the alt attribute used for images, but is distinguished from alt by the fact that alt is meant to be displayed as a *substitute* for an image that cannot be displayed, whereas title *describes* content instead of serving as a fallback for it.

In current desktop browsers, the value of the title attribute is displayed in a tooltip when the associated element is moused over, as shown in [Figure 2-2](#). These tooltips are truncated by some browsers when they run long, though the truncation point varies from one browser to the next.

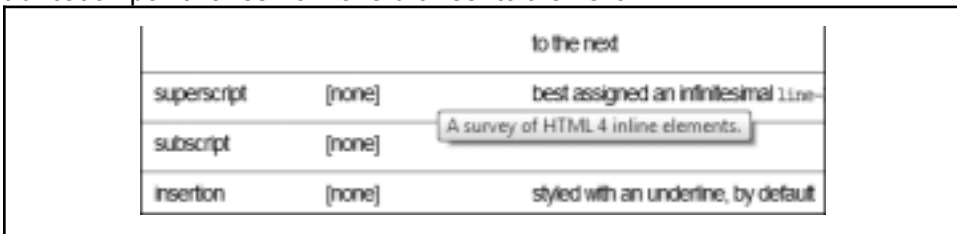


Figure 2-2. A title tooltip, as displayed by Internet Explorer 8 on Vista/Aero

The lang attribute, meanwhile, is a meaningful nod to the “World Wide” part of

the Web. In much the same way that the title attribute can be used to supply supplemental information about the content of an element or the destination of a link, lang does exactly the same for foreign-language content.

Beautiful Parts: Universal Attributes | 15

You are supremely polite when you use the lang attribute, because it aids visitors in the task of understanding the context of foreign-language content even when they cannot make out its exact meaning. Furthermore, screen readers need an accurate lang or Content-Language response header value to pronounce foreign-language content accurately.

The hreflang attribute exists as a counterpart to the lang attribute and is used to signal that a hyperlink points to content written in a language other than that specified for the current document.

Finally, note that XHTML served with the application/xhtml+xml MIME type should use the xml:lang attribute in place of the lang attribute.

When the the lang or xml:lang attributes are used to describe an entire document, they should be attached to the html element, instead of the html element.

The value that you provide for the lang attribute (and the Content-Language HTTP response header field) is chosen from a list composed from various ISO-sanctioned codes and structured according to rules maintained by the IETF.

Table 2-1. Frequently encountered Content-Language values

Language lang/Content-language value

- English en
- English (American) en-US
- English (British) en-GB
- Chinese (Simplified) zh-Hans
- Chinese (Traditional) zh-Hant
- Chinese (Taiwanese, no script specified) zh-TW
- Spanish es
- Japanese ja
- French fr
- Portuguese pt
- Portuguese (Brazilian) pt-BR
- German de
- Arabic ar

Russian ru

Korean ko

16 | Chapter 2: Working with HTML Markup

To learn more about effectively using the title attribute in links, read [“Creating Effective Link Content and title Values” on page 136](#). lang values of interest are listed in [Table 2-1](#).

The contenteditable Attribute in HTML5

The HTML5 specification adds a number of new global attributes to HTML, including contenteditable, which is already supported by most modern browsers. It's mainly intended for providing in-browser rich-text/WYSIWYG editors—the kind of editing interfaces you might find in browser-based blog-authoring tools, for example.

The contenteditable attribute essentially enables you as an author to specify that particular parts of a page (the contents of particular elements) are editable. Within those editable parts of the page, users can potentially perform actions like selecting text, cutting and pasting, and moving text (including by dragging and dropping it), as well as changing the character formatting of text to appear bold or italic, or in a different color, or even actions like adding hyperlinks.

Just setting the contenteditable attribute on a particular element won't cause a browser to actually expose any obvious editing controls to the users. However, it generally *will* enable users to at least perform actions that have common, familiar keyboard shortcuts (for example, Ctrl-X to cut, Ctrl-V to paste, or Ctrl-B and Ctrl-I for bold and italic). Some browsers even provide a text-editing context menu that's available by right-clicking in a contenteditable area; this may add a few additional character-formatting actions that don't have common keyboard shortcuts, such as changing the font size or color of selected text.

It's possible that other browsers will follow suit and expose more contenteditable text editing actions (such as, say, an action for easily adding hyperlinks) through a related context menu. However, all that being said, if you want to provide an in-browser user interface for performing editing actions in contenteditable content, you'll also need to write some programming (scripting) in JavaScript. For example, you can easily add a button to a page allowing users to make selected text bold (rather than forcing them to use a keyboard shortcut), but to make it work, you'll need to add some scripting that associates your button with the action you expect it to perform. (The HTML5 specification provides a number of APIs to facilitate scripting in combination with the contenteditable attribute, but I won't go into the details here.)

Another limitation of contenteditable is that, on its own, it provides no means for users to actually save the contents of any pages they've edited. That's something else for which you, as an author, will need to provide an interface.

Separating Content, Structure, Presentation, and Behavior

Making Your Sites “Safe As Houses”

Imagine a dwelling. The simplest such structure meant to stand for any length of time will have some kind of durable structural frame anchored to the ground and covered with some kind of paneling.

For the client side of a website, that frame and cover are the structural layer, the markup: it defines the overall form of the result.

When you start getting fancy with your house, you can add things like siding, paint, trim, and shingles. This is like the presentation layer of your website, driven on the whole by CSS. In the same way that the walls and roof will fall off of a poorly framed house, CSS will be difficult to use if the markup is not properly assembled.

A good house has things like climate-control hardware, doors, windows, electricity, and plumbing. In many cases, things like this are what make the house truly enjoyable to live in. Likewise, the behavior layer of a website is the part that most clearly responds to user activity. However, without the other parts of the architecture and engineering properly installed, the behavior layer will most likely be ineffective.

And what about the content? Well, as the point of a house is to shelter people and their stuff, so the point of a website is to serve as the ideal vessel of a heap of content. Each HTML page holds a markup structure wrapped around content.

Separation in Practice

When a developer works under the principle of separation, the likely result is that each client-side layer of the site enjoys tremendous independence from the particulars of the other layers. That independence will never be *complete*; at best, it will enable in new properties the reuse of assets that already exist. At any rate, the principle of separation assumes the following dependencies:

1. A site's behavior loses its punch without the presence of an effective presentation.
2. A site's presentation is dependent upon the underlying quality of its structure.
3. Without thoughtfully assembled content, creating a solid site

structure becomes a fool's errand.

However, it's easy to achieve a level of “layer independence” that will minimize the impact of changes—so that, by defining a class that is only assigned to an element when a visitor interacts with it, you can make unlimited changes to the presentation of an element without also needing to alter any JavaScript that defines its behavior. Like wise, you can give yourself free rein to fiddle with a stylesheet and completely revise a site's presentation without being forced to dig into any of the site's structure or

18 | Chapter 2: Working with HTML Markup

content—the “CSS Zen” approach (see the section [“The Functional Principles of CSS Zen” on page 60](#)).

Working with Document Trees

Much of the work at the beginning of a website development project revolves around developing a simple HTML structure that CSS and perhaps eventually JavaScript can use as a framework for their presentation and behavior activity. This work focuses on creating a basic structure that many documents can use as their foundation and possibly as their template. At this point in the process, the focus is less on content and more on common structures that wrap that content. [Example 2-2](#) shows the body element of a simple HTML document structure.

Example 2-2. A simple HTML document structure

```
<body>
<h1>...</h1>
<div id="main">
<div id="priorityContent">
<div id="bodyCopy">

<h2>...</h2>
<div class="section">
...
</div>

<h2>...</h2>
<div class="section">
...
</div>
</div>

<div id="sidebar">

...
</div>

<ul id="primaryNav">
...
</ul>
<div id="footer">
```

```

<ul id="secondaryNav">
...
</ul>
<p id="colophon">...</p>
</div>

</div>
</div>
</body>

```

Separating Content, Structure, Presentation, and Behavior | 19

This sample provides more than just a template. Its nested and labeled elements also define a structure that CSS (and JavaScript) can build on, creating a tree (labeled using # for ids and . for classes) that looks something like:

The production standards enforced at many workplaces generally rely on class values in templates more heavily than suggested in [Example 2-2](#) and the document tree outline below.

- body
 - h1
 - div#main
 - div#priorityContent
 - div#bodyCopy
 - h2
 - div.section
 - ...
 - div#sidebar
 - ...
 - ul#primaryNav
 - div#footer
 - ul#secondaryNav
 - p#colophon

Sketching your document tree as a nested list and then creating markup from the structure you've defined may make it easier to see and manipulate the tree structure you'll need. The tree shown here is relatively simple, and doesn't reach down into individual paragraphs or other forms of content. As you fill documents with additional content, you will be constantly extending the document tree, and expanding the amount of material that your stylesheets and perhaps your JavaScript code can modify.

Browsers, Parsing, and Rendering

Current web browsers typically parse and render content piecemeal, quite often starting the process before a page has been received in full by the browser. HTML browsers— or more generically, user agents—will process an HTML or XHTML document serially from the beginning of a document’s source, working out the relationships between the various elements it contains and filling in gaps if necessary to create a document tree. Meanwhile, they read any CSS specified by the HTML in a similar fashion, matching

20 | Chapter 2: Working with HTML Markup

up stylesheet selectors to the elements contained in the page as described in the next chapter.

The serial nature of these processes is important for three reasons:

The only way in which user intervention can affect parsing is to halt it The markup, CSS, JavaScript, session data, and user data received by the browser in the scope of a single page set the stage for everything that happens until the page is completely rendered.

Until a page and its related media are completely received, parsed, and rendered, their appearance is subject to change at the hands of the browser’s rendering engine In high-latency environments, slow arrivals can create visually disconcerting results as the page shifts over time to accommodate recently arrived components. This behavior can lead to the dreaded “Flash of Unstyled Content,” which is explored briefly on this [book’s companion website](#).

There is no strict rule as to what a web browser should or should not parse, as long as the data in question can be interpreted

Browsers are permissive about what they attempt to download and parse. This permissiveness leaves it to the discretion of a site’s developers to conserve resources, a task best accomplished by ensuring that stylesheet data is matched with care to the requirements of the current document. This is a potential cause for concern to those who need to account for performance in marginal environments.

Because Ajax uses the W3C Document Object Model Application Programming Interface (DOM API) to update the contents of arbitrary elements within a page, it’s absolutely vital that markup within any page intended to contain the output of Ajax calls be syntactically correct. This is particularly true with respect to sibling elements of those meant to be updated.

Markup syntax errors alter the document tree and element boundaries into a configuration different than what’s intended, which can make it unnecessarily difficult to find the cause of JavaScript errors in Ajax-oriented code.

Dynamic HTML, Ajax, and Rendering

First- and second-generation desktop browsers only ran a single set of rendering passes per page request, so that additional rendering would only take place after another page request had been sent to a server. All subsequent desktop browsing platforms have made it possible to insert additional content after the initial “page load,” a feature that was called “Dynamic HTML” until the XMLHttpRequest API became popular. The ability of this new API to asynchronously request and insert new content without loading an entirely new page eventually led to the adoption of the term “Ajax.”

Browsers, Parsing, and Rendering | 21

CHAPTER 3

CSS Overview

Like in a cinematic or musical work, the “Good Parts” of CSS are easier to find if you have a basic understanding of what’s going on. This chapter lays the foundation for what’s to come in the rest of this book—it explores the role of CSS in creating successful websites, and provides a survey of its basic components.

If you’re in a hurry to get to the Good Parts, you can skip ahead. There are some mentions of Bad Parts and Awful Parts here worth noting, though, and CSS is complicated enough that a quick review can be helpful.

Connecting Stylesheets to HTML Documents

HTML documents can specify the stylesheets that are applied to them, using the link element, the style element, or the @import declaration.

Go to <http://www.htmlcssgoodparts.net/> for an interactive demonstration of the relationships between stylesheet rules and elements in a typical page.

Referencing a Stylesheet with link

The most common method of associating styles with your document is to use a link element within the head of a document. The source of link elements usually looks something like this:

```
<link rel="stylesheet" href="/styles.css" media="screen" title="Primary Stylesheet" />
```

This approach also supports stylesheet *choices*: a stylist can create multiple stylesheets, assign a title to each, and assign a second rel (relation) value of alternate to all but one of the referenced stylesheets. (Multiple rel values should be separated by spaces.)

23

Users will then be able to choose which stylesheet they want to associate with your site. This feature is supported by Firefox, recent versions of Safari, and Internet Explorer 8.

Targeting Internet Explorer Versions with Conditional Comments

Internet Explorer's partial support for CSS has created a variety of problems for developers. However, one of Internet Explorer's other nonstandard features makes it possible to specify stylesheets *only* for Internet Explorer, even to the degree of specifying particular versions of that browser.

Updates made to Windows in late 2009 disabled Internet Explorer 8's support for conditional comments when operating in "IE8 Standards" mode.

Internet Explorer defines HTML comments differently than other browsers, which allows you to include source that only Internet Explorer will parse as proper markup.

One possible example that references a stylesheet is:

```
<!-- [if lt IE 8]><link rel="stylesheet"
href="/styles.ie.css" media="screen" /><![endif]-->
```

The <![endif]--> closing "tag" is a constant feature of such markup. The opening matter is written in the following format, with user-supplied values in emphasis:

```
<!-- [if version_constraint IE version]>
```

The user-supplied values work as follows:

version_constraint

This item is optional, but where present can take one of four

forms: • gt: greater than [>]

• gte: greater than or equal to [≥]

• lt: less than [<]

• lte: less than or equal to [≤]

version

This item is also optional, and where used corresponds to a major release: 5, 5.5, 6, 7, or 8.

The Internet Explorer conditional comment syntax also supports Boolean AND, OR, and NOT operators, which are explained in more detail at the [Microsoft Developer Network site](#).

Low-pass and high-pass rule filters can serve as fit alternatives to stylesheets conditionally targeted to legacy versions of Internet Explorer. These rule filters are discussed in more detail in [Chapter 14](#).

24 | Chapter 3: CSS Overview

Replacing link with style

When the style element is used, it can contain any quantity of valid CSS. One effective use of style blocks is to serve CSS rules that are specific to a single page; I make a regular habit of using this technique. Many developers use style elements to reference stylesheets requested via the @import rule, which is discussed in more detail next.

The conditional comments described earlier can also enclose a style element or its content.

It's generally best to keep style blocks short, as their presence affects the proportion of keyword-rich content in a page, thus having the potential for a slightly negative impact on Search Engine Optimization (SEO) efforts.

When a page marked up in XHTML is served with the correct MIME type (application/ xhtml+xml), style content must be placed inside a <![CDATA[...]]> (character data) block.

Using @import

The @import statement first became popular in the late 1990s, when developers discovered that Netscape 4 wouldn't parse it. This made it easy to include more advanced stylesheets that would work with other browsers, while leaving Netscape 4 alone.

In contemporary use, `@import` declarations are reduced to their original intended function, which is to serve as an analog to an `include` function that is applied specifically to stylesheets.

`@import` declarations must always appear at the top of a stylesheet's source order, valid `@import` declarations can only be preceded by an `@charset` declaration. The browser parses and applies the styles in an `@imported` stylesheet as though they were in the place of the `@import` declaration that referenced them, a fact that can affect rule priority.

For the sake of consistency with other bits of CSS syntax, only the parenthetical method of reference will be demonstrated:

```
@import url(/form_styles.css);
```

It is also possible to apply stylesheets called `@import` to specific media, which is discussed shortly.

Beware of style Attributes!

The first rule of standards-friendly development (see [“Rules of Standards-Friendly Development” on page 46](#)) demands that you keep presentation details out of your markup, so avoid the style attribute however and whenever possible. In those

Connecting Stylesheets to HTML Documents | 25

(extremely) rare instances where it *must* be used (for example, Content Management Systems that lock out stylesheets), it should contain the desired series of valid property/ value pairs, just as if those same pairs were being included in a stylesheet rule applying only to that element.

The style attribute is further discussed (and subjected to passionate abuse) in [“The Awful Parts” on page 286](#).

Targeting Rules to Specific Media

HTML and CSS allow you to create different stylesheets for different media, most commonly screen and print. A single document might have several stylesheets, each targeted at one or more media. There are three approaches to applying styles to specific media:

Add an optional media attribute/value pair to an appropriate link or style element

Adding a media attribute to one of these elements will cause the valid rules contained within that element's scope to apply only to the desired media.

Therefore, if you want a linked stylesheet to apply only to printed pages, you would include `media="print"` in the applicable link tag.

Add an @media block to a style block that hasn't already been assigned a mutually exclusive media value

For example, `@media print { body { font-size: 12pt; } }` will cause the default type size of a given page or site to be changed to 12 points.

Add a media value to an @import declaration that isn't already placed within a mutually exclusive media scope

In the same way that the @media selector trails with the names of one or more recognized media, the file reference in an @import declaration can be followed with the names of one or more recognized media, for example, @import(/styles.print.css) print;. This approach targets all of the rules in that stylesheet to the desired medium or media.

Note that @import declarations placed *inside* @media blocks are invalid.

Where multiple media are named, they should be comma-separated.

The following media type values are described in the CSS 2.1 specification and claim greater-than-insignificant support from browser and other user agent vendors:

all

All devices

26 | Chapter 3: CSS Overview

screen

Monitor-type displays attached to personal computers, typically cathode ray tube (CRT; “TV-type”) or liquid crystal diode (LCD; “flat panel”) displays; often applicable mobile device displays as well, at vendors’ discretion

print

Paper sheets of arbitrary number and area, coated with ink, pigment, or toner

handheld

Mobile devices and personal digital assistants (PDAs); poorly supported by all but very recently marketed devices, as of 2009

projection

Tabletop projectors; poorly supported by nearly all vendors

speech

Screen reader and text-to-phone platforms; poorly supported

The remaining media types described in the CSS 2.1 specification are functionally unsupported:

braille

Braille terminals

embossed

Braille printers

tty

Two-dimensional fixed pitch display environments (usually a monochrome

CRT display or command-line client software); accurately cognate to the traditional Unix designation for dumb terminals

tv

Television browsers, like the erstwhile WebTV

Choosing the Elements You Want to

Style: Writing Selectors

A typical stylesheet, regardless of its scope, is a series of rules structured as follows: selector { property: value; property: value; [...] }

The bad news—and the bane of many newcomers to CSS—is that this structure is terse to the point of impenetrability.

The steep learning curve of CSS syntax is rewarded with the ability to affect a page's presentation with a superlative degree of granularity. Any selector can point to any arbitrary set of elements within a page, and CSS properties can accomplish anything within the limits of an implementer's experience and imagination, when put to thoughtful use.

Choosing the Elements You Want to Style: Writing Selectors | 27

The concepts explained briefly here are taken up in much greater detail in [“Applying Taxonomy Through the Cascade” on page 70](#).

Parents, Children, and Siblings: Element/Node Relationships

The section [“Tags, Elements, and Attributes” on page 8](#) introduced the idea of element *nesting* for the purpose of explaining how tags-inside-tags need to be written. Element nesting opens the door to one of the most important aspects of applied HTML and CSS:

It is not only allowed but actually encouraged to “wrap” stretches of clearly related content in elements set aside for just that purpose, and to assign descriptive ids and/or classes to such wrappers.

When such “semantically appropriate” elements are used to enclose content, new relationships are created in the document tree, thereby increasing the number of CSS selectors that can be used in the course of implementing a design.

Multiple nested elements have what are referred to as *parent*, *child*, and *sibling* relationships:

Document tree

The notional branching structure of all elements in a document.
Synonymous with “Document Object Model” as applied to a specific document.

Parent

The element that directly contains the element at the focus of concern. *Ancestor*

An element higher in the document tree, possibly many levels higher, that contains the element in question.

Child

The element that is directly contained by the element at the focus of concern. *Descendant*

An element contained by the element in question which is deeper in the document tree.

Sibling

An element that shares a common *immediate* parent with the element at the focus of concern.

CSS makes a clear distinction made between generic parent and child elements, and those that are *immediate* or *direct*. For example, an `li` element in a valid document claims some `ul` or `ol` element as its direct parent, but will also have at least one—if not two—other parent elements within its document tree: `body` (which is required by the DTD for the various flavors of HTML 4.x) and quite likely another block element.

28 | Chapter 3: CSS Overview

Simple Selectors

Typically, selectors interface with markup at three main points: element names, class attribute values, and id attribute values.

Elements

`p { ... }`

classes

`.about { ... }`

ids

`#corporatehistory { ... }`

The following fragment of markup includes hooks for all of the example selectors just shown:

```
<body class="about" ... >
...
<div id="corporatehistory" ... >
...
<p>The 1990s were a time of drastic change throughout the industry.<p> ...
</div>
...
```

</body>

The `p { ... }` selector will apply to the `p` element in code fragment just shown. The `.about { ... }` selector will apply to the body element, whose class value is "about". And finally, the `#corporatehistory { ... }` selector will apply to the div element with an id of "corporatehistory".

Beyond these three foundations, CSS 2.1 specifies other selector types, including universal selectors (`*`), child selectors (`div > p`), descendant selectors (`div p`), adjacent selectors (`ol + p`), and attribute selectors (`p[lang]`, `p[lang="en"]` or a number of other variants). It also includes the `:first-line`, `:first-letter`, `:before`, and `:after` pseudoelements, as well as a variety of pseudoclasses: `:first child`, `:link`, `:visited`, `:active`, `:hover`, `:focus`, and `:lang`. CSS3 adds even more pseudoclasses and pseudoelements, but is still in development.

Multiple and Descendant Selectors

The capacity to combine multiple selectors in a single rule is by far the greatest contributor to the versatility of CSS. Selectors can be combined and comma-separated to apply the same characteristics to multiple arbitrary elements, whitespace-separated to reference child elements, and concatenated to enforce a high degree of granularity. There is no limit on the number or type of selectors that can be associated with a single stylesheet rule.

Choosing the Elements You Want to Style: Writing Selectors | 29

The examples shown in [Table 3-1](#) are assumed to be in a stylesheet that applies to an entire site.

Table 3-1. CSS selectors scoped in plain English

Selector	Applies to
<code>p</code>	All paragraphs in the document
<code>.about</code>	All elements in the document with a class value of about
<code>#corporatehistory</code>	The element in the document with an id value of corporatehistory (if present)
<code>h1,h2,h3</code>	All first-, second-, and third-level headings in the document
<code>.privacy,.copyright</code>	All elements with a class of privacy or copyright
<code>#header,#footer</code>	The element assigned an id of header, and the element assigned an id of footer
<code>p.footnote</code>	All paragraphs assigned a class of footnote

#bodycopy.usergenerated	An element that has been assigned both an id of bodycopy <i>and</i> a class of usergenerated
.navigation a	All links with an ancestor parent assigned a class of navigation
#primarynavigation li.current	All list items with a class of current and an ancestor parent with an id of primarynavigation
.about #bodycopy	Any element on the site with an id of bodycopy and an ancestor parent assigned a class of about
body#personalproducts, body#proproducts, body#enterpriseproducts	The body elements within the site assigned the ids personalprod ucts, proproducts, and enterpriseproducts
body#personalproducts #bodycopy, body#proproducts #bodycopy, body#enterpriseproducts #bodycopy	The elements assigned an id of bodycopy, within the documents sug gested by the previous example
ol li ol li ol li	A list item in the third level of a nested ordered list (such as an outline)

Selecting Direct Child Elements

CSS provides the > selector to create selectors for elements with an immediate child relationship, so that:

```
#bodycopy>p { ... }
```

refers to the paragraph element in:

```
<div id="bodycopy"><p>...</p></div>
```

30 | Chapter 3: CSS Overview

but *not* the paragraph element in:

```
<div id="bodycopy"> ... <blockquote><p> ... </p></blockquote> ... </div>
```

The > selector is discussed as one of the Bad Parts—not because there’s anything wrong with it, but rather because it’s not supported by Internet Explorer 6.

Rule Conflicts, Priority, and Precedence

The *cascade* allows well-written selectors to target *any* range of elements in the document, without respect to the level of the document tree in which that range of elements lies.

A close look at the selector examples presented so far reveals that conflicts seem inevitable. A rule such as `p { ... }` would apply to the preceding source example, but presumably so would `#bodycopy p { ... }`. When there is a conflict, which value gets applied?

Selector Priority

The types of selectors used in a rule dictate that rule's priority. In *ascending* order of weight, they are:

1. User agent stylesheet selectors
2. User stylesheet selectors
3. The universal selector (*)
4. Elements and pseudoelements (e.g., `first-letter`)
5. Classes, pseudoclasses (e.g., `:hover`), and attributes
(`[selected="selected"]`)
6. ids
7. Values of inline style attributes, as explained in [“The Awful Parts” on page 286](#)

Given any two rules, the one with the highest-priority selector will automatically take precedence. In cases where two rules contain selectors of equal priority, it then becomes necessary to count the number of selectors in each rule—a rule with two id selectors takes priority over a rule with one id selector and four (or eighteen) class selectors, for example.

When any two selectors claim identical priority and weight, it then becomes necessary to consider the presence of any important values that they contain, as well as their relative position in the source order of the styles applied to a document. The importance of rule source order is explained in the following section, and taken up directly in the explanation of link pseudoclasses (see [“Link Pseudoclasses” on page 137](#)).

Avoiding Rule Conflicts

Where two conflicting rules claim an *identical* priority, the browser applies the *latter* of those two rules, considered in terms of style source order. Style source order is determined by the order in which external stylesheets and style element content are inserted into a document, so that given the following fragment of markup:

```
<link rel="stylesheet" type="text/css" href="/styles.css" media="all" /> <style
```

```
type="text/css">@import (/styles.signup.css);</style>
```

The contents of `/styles.signup.css` are assigned a later source order than the contents of `/styles.css`. Placing the `@import` declaration in `/styles.css` would instead give the contents of `/styles.signup.css` an *earlier* source order.

There are two other methods of increasing the priority of property/value declarations: the `!important` value and overloaded selectors. The former is best applied to general cases, while the latter is best applied to edge cases.

The `!important` value follows a normal property/value pair, like

```
SO: color: #f00 !important;
```

In practice the values supplanted with `!important` are given absolute priority in all stylesheet rule conflicts, unless a user stylesheet also addresses the same conflict with an `!important` value of its own.

In its turn, selector overloading relies on the fact that *all* selectors attached to a rule contribute to that rule's priority, even if the elements they specify are completely absent from the document to which the styles are being applied.

Consider the following two rules in the context of the same page and stylesheet:

```
a:visited { color: rgb(128,0,128); }  
.sidebar a { color: rgb(0,0,255); }
```

Given the requirements of the specification, both rules acquire the same priority. To have any hope of being applied, the second of the two rules just given *must* be inserted later in the stylesheet. However, if project requirements deny that outcome, it might be practical to overload the rule with an additional *bogus* selector.

This scenario is fairly unlikely. Instead, selector overloading demonstrates its greatest value when a rule assigns style values to several sections of a site or page at once, and in so doing makes it otherwise impossible to give priority to values in even narrower contexts.

Overloaded selectors should be embellished by accompanying comments whenever possible, in order to make them more accessible to automated search/replace functions.

32 | Chapter 3: CSS Overview

Value Inheritance

Only `font/text` and foreground color values are preserved within an element's descendants. Background properties often *appear* to be inherited, and this illusion works because where two elements share a context and overlap in the layout—as is always the case with descendant elements that share a stacking

context. The latter element in the source order is always stacked *above* its immediate parent or other ancestor.

Element stacking contexts are discussed in greater detail in [Chapter 6](#).

Meanwhile, there are two types of elements that do *not* inherit the font/text and fore ground color values of their parent elements:

Form controls

The type in form controls is set in the fonts and colors specified by the operating system defaults, unless it is deliberately reset in the stylesheet.

iframes

Since an iframe is populated by its own separate document, its font/text and fore ground color properties are defined by stylesheet rules associated with that document.

CSS Property and Value Survey

Figuring out how to specify the element you want to style is probably the hardest part of learning CSS and writing production stylesheets. However, climbing that learning curve doesn't lead to results until you know how to supply style properties and values to the elements that you've carefully chosen.

CSS Units

While CSS supports a seemingly endless list of properties, the scheme for setting values is fairly predictable. [Table 3-2](#) describes the most frequently used values.

Table 3-2. Commonly encountered CSS length/size, keyword, and

color units **Unit Type Example**

px (pixels) length width: 744px;

em (ems) length margin-left: 1.25em; % (percent) length left: 34%;

pt (points) length font-size: 12pt; **CSS Units | 33**

Unit Type Example

in (inches) length margin-top: .75in; cm (centimeters) length margin-top: 1.905cm; xx-small

... xx-large font size font-size: large; rgb(r,g,b) color (decimal) background-color:

rgb(221,204,187); #rrggbb color (hexadecimal) background-color: #ddccbb; #rgb color

(hexadecimal, reduced depth) background-color: #dcb;

Cross-Media Length and Size Units

There are three commonly used units in stylesheets intended for screen display:

px (*pixels*)

Pixels are absolute units, equal to one pixel on the user’s screen display; always expressed as an integer.

em (*ems*)

In digital typesetting environments (including CSS), an em is equivalent to the greatest possible height of a glyph (letter) in the applicable font and size combination. The contemporary definition contrasts with the *historical* definition: the width of a capital “M” in the font and size of the type to which it is applied as a measurement. This unit is usually expressed with a floating-point value.

% (*percent*)

Percentage units are computed relative to some baseline measurement, which varies according to property and context. Floating-point percentage values are allowed.

em and % units are discussed in [“Layout Types and Canvas Grids” on page 106](#).

Pitch and the Value of a Pixel

The screen, handheld, and projection media types all support the px unit, but CSS provides no standard mechanism for defining display pitch.

Since the pixel is the atomic unit of screen displays, all page elements must be computed in terms of pixels before rendering can take place. Because of this requirement, display pitch will determine the literal size of everything presented by the browser. The smaller the display pitch, the smaller everything will be when a page is rendered and displayed.

[Table 3-3](#) shows the approximate arithmetic behind common display resolutions and pitches, as found on both contemporary backlit LCD and older CRT displays. Commonly encountered form factors are shown in bold.

34 | Chapter 3: CSS Overview

Table 3-3. Commonly encountered screen display form factors, resolutions, and

display pitches	Size (viewable)	Resolution	Aspect ratio	Width (mm)	Height (mm)
Pitch (mm)	px/in	10.2"	1024×600 ≈17:10	224	130
		0.218	116	12"	(CRT) 640×480
	4:3	244	183	0.381	66
	12"	1024×768	4:3	244	183
		0.238	106	13"	1280×800
	16:10	280	175	0.219	116
	14"	(CRT) 800×600	4:3	284	213
		0.355	71	15"	1024×768
	4:3	305	229	0.298	85
	16"	(CRT) 1024×768	4:3	325	245
		0.318	80	17"	1280×1024
	5:4	337			

270 0.263 96 17" 1366×768 ≈16:9 376 212 0.276 92 17" 1440×900 16:10 366 229
0.254 100 **19" 1440×900 16:10 409 256 0.284 89** 22" 1680×1050 16:10 474 296
0.282 90 23" (CRT) 1600×1200 4:3 447 335 0.279 91

As of this writing, two types of displays (19", 1440×900; and 22", 1680×1050) are particularly high sellers on Amazon.com (though both types are outsold by netbooks). Going by this table, the pitches of these two types of displays differ by only 2µm (less than 1%), averaging out at nearly 90 pixels per inch.

Considering that Windows assumes a display pitch of 96 pixels per inch for the purpose of displaying print documents onscreen, it would appear at first that stylists have little to worry about when it comes to predicting the literal size of their product on the screen display of the typical user.

In most cases, default assumptions offer few caveats to trouble the conscientious stylist. However, consider the fact that a pixel on a netbook contains less than two-thirds of the area of a pixel on one of the high-selling displays mentioned earlier. This means that the physical dimensions of a layout will decrease by one quarter on each axis when viewed on a netbook, as compared to the most popular types of desktop monitors.

The physically variable nature of pixels leads to a concept that every web developer would do well to keep in mind:

Designers who fail to account for the range of conditions under which their sites are visited will likely experience shock when exposed to their work in an unexpected setting. Stylists are among the reviewers who can prevent such surprises.

Print-Friendly Length Units

[Chapter 1](#) pointed out that unlike its predecessors, the Web is an unbounded medium. That lack of constraints applies not only to the Web's domain of information, but also to its interface—when viewed on most hardware platforms, web documents can be scrolled, minimized, maximized, and otherwise manipulated within browser windows to the limits of the designer's imagination and the visitor's patience.

The print medium yields with greater ease to assumptions about environment, since readers of printed pages typically read from sheets of US Letter (8½" × 11") or ISO A4 paper, which are similar in size.

Print stylesheets can make good use of additional units, all of which can be specified in floating-point values:

pt

Type is traditionally measured in *points*; there are 72 in an inch. Expressed in Système International (SI) units, a point is roughly equivalent to 353 µm, yielding slightly less than 28.35 points per centimeter.

in

One inch is defined by international treaty as being equal to 2.54 cm. By way of comparison, a row of four hexagonal wooden pencils laid side by side and flush will measure nine-eighths of an inch (1.125") across.

cm

Centimeters. One sheet of A4 paper measures 21 cm × 29.7 cm.

This list is not comprehensive; additional units are described on this [book's companion website](#). Also, note that line-height values can be specified with floating-point numbers and without a unit. In such cases, a value of 1 is equivalent to 1em.

font-size Keywords

In situations where the priority of accessibility outweighs the need to express design composites precisely, font-size keywords can be used to surrender presentation control to the visitor. The domain of font-size keywords contains seven values, listed below from largest to smallest:

- xx-large
- x-large
- large
- medium (default)
- small
- x-small
- xx-small

36 | Chapter 3: CSS Overview

font-size keyword values are also discussed in [“Size Keywords” on page 217](#).

Color Units

CSS supports a three-channel (red, green, blue) color space with eight bits of color depth per channel. Such a space yields 16.7 million (2^{24}) possible colors.

Wherever there's a need to reference a color, stylists have at their disposal three styles of notation:

rgb(r,g,b)

Three channels, decimal; each channel takes a range of 0–255.

#rrggbb

Three channels, hexadecimal; each channel takes a range of 00–ff.

#rgb

Three channels, hexadecimal, reduced depth; each channel takes a range of 0–f. The equivalent color in full-depth notation is found by doubling each hexadecimal digit, so that #6cf and #66ccff are identical.

When creating stylesheets, it's important to choose the most appropriate color notation and use it with ironbound consistency for the sake of stylesheet legibility. The advantages and disadvantages of each are described in [Table 3-4](#).

Table 3-4. Advantages and disadvantages of the three styles of CSS

color notation	Style	Advantages	Disadvantages
Six-digit hex	• Precision		• Difficulty of visualization
	• Ease of migration from legacy markup		• Illegibility
24-bit decimal	• Human readability		• Vulnerability to input errors
	• Accessibility to scripted transitions		• Lack of copy+paste functionality in third-party tools
Truncated hex	• Simplicity		• Lack of depth
	• Suitability for prototyping		

[Chapter 9](#) goes into greater detail about working with color.

Key CSS Layout Properties

In order to implement all but the simplest layouts, it becomes necessary to use a number of layout properties that alter the flow of elements within a document. The most functionally useful properties and values are described in [Table 3-5](#) (defaults are in bold). These properties and values will be explained in greater detail in [Chapter 6](#).

Key CSS Layout Properties | 37

Table 3-5. Commonly supported CSS layout properties and values

Property Values

display • block

- inline
- inline-block
- none

width/height • [length]

- auto

float • left

- **none**
- right

clear • **both**

- left
- none
- right

position • absolute

- fixed
- relative
- **static**

top/right/bottom/left • [length]

The *functions* of the properties listed in [Table 3-5](#) are described next and shown in [Figure 3-1](#).

display

HTML specifies that elements exhibit one of several kinds of layout behavior. Normally this behavior is set by the element's definition in the DTD, but that behavior will correspond to and may be overridden by the value of the CSS display property. inline describes elements that flow without deliberate line breaks on a (usually) common baseline, and have limited interaction with CSS layout and box properties. block describes elements that are followed and preceded by linebreaks, and expand to fill the entire width of the containing element (unless otherwise specified). inline-block describes elements that flow like inline elements, but interact with the full range of CSS layout and block properties, like block elements. The none value is cognate to elements that are hidden by default, and supplying this value will remove the affected element from the document flow entirely.

width *and* height

Describes the dimensions of block and inline-block elements. A width value of auto causes the affected element to expand to the full width of its containing

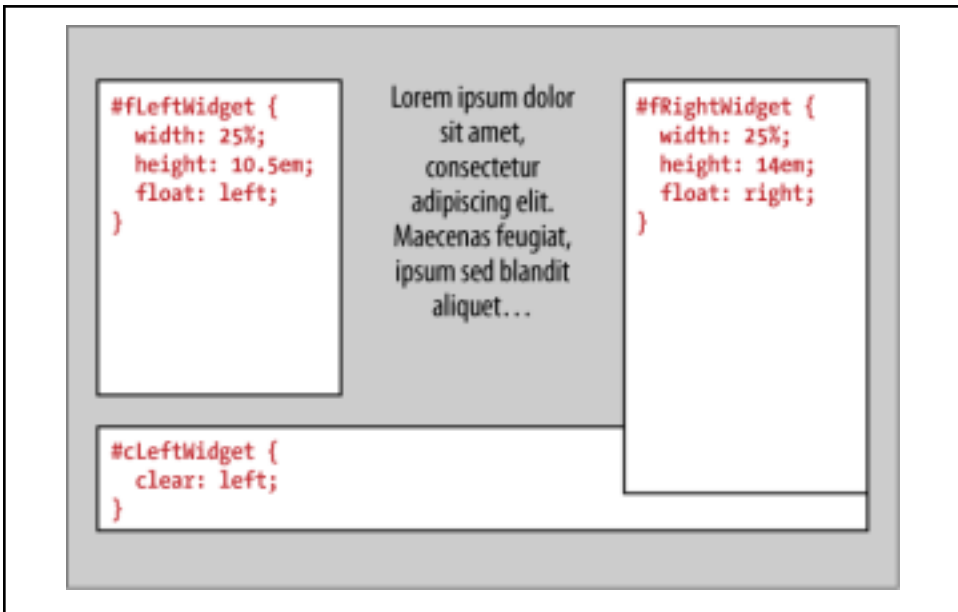


Figure 3-1. The use of the `float` and `clear` properties changes the relationship between the affected element and the element(s) that follow it

element, while a height value of `auto` causes the affected element to expand to the full height of its *nonfloated* content.

`float`

When changed, specifies that an element should hew to the specified margin of its containing element, and allow following elements to flow around it. A float prop erty/value pair must always appear in tandem with an explicit width property/value pair, unless it's assigned to an element (such as an image) that has an *intrinsic* width.

`clear`

Describes the containing element margin(s) to which an element should be anchored, thus ensuring that it will be placed *below* any nearby preceding elements that have a comparable float value.

position, top, right, bottom, *and* left

Alters the location of the specified content border(s) of the affected element, when a position value other than `static` is supplied. As layout tools go, these properties are spectacularly powerful—powerful enough that they're explained at exhaustive length in [“CSS Positioning Properties” on page 96](#). [Figure 3-2](#) describes the re lationships between various types of positioned elements.

The layout properties described here are a small subset of the full body of CSS layout properties. The others are described throughout the rest of this book.

```
body>div {  
  position: static; }
```

```
body>div>#abs {  
  position: absolute;  
  left: 32px; top: 32px; }
```

```
body>div>#relDiv {  
  position: relative;  
  top: -132px;  
  left: 36px; }
```

```
body>div>#relDiv>#absInRel {  
  position: absolute;  
  left: 288px;  
  top: -120px;  
  width: 216px;  
  height: 216px; }
```


Figure 3-2. A position value other than static changes the positioning context, as shown

CHAPTER 4

Developing a Healthy Relationship with Standards

The Web as practiced is not the same as the Web as specified. While the Web is built on standards and specifications, adherence to the rules is sporadic at best. Software developers often extend functionality, leave out functionality, or implement things incorrectly. Web developers often create sites that more or less conform to the specifications, but they may also tune their work to run on a particular browser, with all of its idiosyncrasies, or leave their slightly broken markup at the mercy of a browser's rendering engine.

Ignoring or abandoning standards to focus exclusively on browser results leads to a maze of constant testing on every imaginable browser, and possibly a powerless sense of resignation when you recognize the inevitability of alienated site visitors. On the other hand, sticking to the spirit, the letter, and the fine details of specifications may limit your ability to reach visitors through the tools they have, rather than the tools you wish they had. Fortunately, there is a middle way.

The Broad Landscape of Web-Related Standards

While HTML and CSS are the focus of this book, there are a number of significant standards that relate directly to web development. These include:

HTTP 1.x

Hypertext Transfer Protocol—already explained in brief earlier—claims both the W3C and the IETF as custodians. This partnership is appropriate, as the IETF is the body responsible for the ongoing development of the protocols used on the Internet as a whole. The [appendix](#) outlines aspects of HTTP that HTML and CSS developers should understand.

Web Content Accessibility Guidelines (WCAG)

Two versions of the Web Content Accessibility Guidelines have been published. The first became a W3C Recommendation in May 1999, and is referenced directly

41

in the United States Government regulations that define websites that are accessible to the functionally impaired. The latest version achieved Recommendation status in December 2008, and applies not only to traditionally designed websites, but web platforms in general.

The syntax, grammar, and core objects of the language usually referred to as JavaScript are defined in a standard endorsed by Ecma International (originally chartered as the European Computer Manufacturers Association).

Document Object Model (DOM) Levels 1–3

Valid web documents have a coherent, tree-like structure, and the DOM specifications define the programmatic interfaces to that structure.

The vendor-specific DOM APIs used when Dynamic HTML was first supported are sometimes referred to as “DOM Level 0.” Levels 1 through 3 are defined in a number of W3C Recommendations and Drafts.

ISO 639, 8859, and 10646

The W3C and vendors currently rely on these standards for implementing character encodings and language references. The last of these is called Unicode, and is encoded using one of the Unicode Transformation Format (UTF) schemes. Nonlogographic writing is often encoded using UTF-8. For more information about character encoding, consult [“Character Encoding in Brief” on page 224](#).

The Web also relies on standards for things like image formats (JPEG, GIF, PNG, SVG) and other content that might be included in pages.

Why Web Standards?

Since the publication of the first IETF draft specification for HTML, browser vendors and site developers have made a frequent bad habit of disregarding published web standards. At the same time, the community of developers who make a point of respecting those standards (of which this author considers himself a devoted if usually quiet member) has never been anything but vocal and predictable, if not actually disciplined. There are a number of issues at work behind the scenes of the ongoing debate.

This section addresses web standards as they are typically promoted.

Interoperability

Untested assumptions about visitors are a big mistake. Common adherence to standards would reduce the number of assumptions. Developers could build their sites and deploy them with a minimum of platform testing. And who doesn’t want that?

42 | Chapter 4: Developing a Healthy Relationship with Standards

Market Forces

The virtues of interoperability do not, however, harmonize easily with the hot desire for bells, whistles, and pretty things often felt by artists and marketers.

Browser vendors cannot ignore the imperative to innovate, and the market usually works on a shorter life cycle than the standards acceptance process.

Market forces are what drove the prospect of common standards compliance off the rails in the first place. In early 1995, table support was introduced in Netscape 1.1, while codification of earlier enhancements brought to market by Mosaic 2.0 was still awaiting acceptance. In effect, Netscape—then still deserving of the “startup” label and two years from its groundbreaking entry into the equity markets—was capable of running circles around the standards adoption process, and the market responded. The resulting conditions nurtured a diffident attitude toward web standards that persists more than a decade later.

Forward Compatibility

It is often argued that standards compliance ensures the longevity of sites that respect it; while features are often added to user agent platforms, they are rarely removed or disabled (the blink element being a notable but absurd exception). Older standards, meanwhile, tend to lie within the lowest common denominator of features supported by all user agents. The upshot of these two facts is that standards compliance allows sites to better survive across browser versions.

Accessibility

Standards compliance tends to make materials more accessible to impaired users, many of whom rely on various forms of third-party assistive technology to ensure a meaningful experience. The standards are a useful guide in this respect for a number of reasons:

- While not enforced by an impartial body, W3C Recommendations *are* authoritative and as such are incorporated into requirements of statute law relating to accessibility, especially in the United States.
- The published Recommendations provide vendors of assistive technology with baseline expectations for their customers’ browsing environments, even when that baseline follows lowest common denominators.
- From the beginning, one of the principal design goals of HTML has been cross media compatibility, which makes it easier to create technology.

Vendor Priorities

After the release of Internet Explorer 6 in 2001, Microsoft’s attention to the web user experience entered an era of somnolence that has only just definitively ended with the release of Internet Explorer 8, a substantial upgrade.

IE6 was released at a time when market shares of competing user agent platforms were on the wane, and the public understanding is that Microsoft made a strategic decision to rest on its laurels until developer community outcry—driven by an official United States Government recommendation to stop using Internet Explorer altogether, among other factors—encouraged it to resume significant ongoing development of Internet Explorer.

A similar incident illuminates Netscape 4's poor support for CSS. When CSS 1.0 was in development, Netscape and Microsoft offered competing proposals to the W3C, and Netscape's proposal was rejected outright. It's apocryphally understood that because Netscape 4 was about to pass its RTM (release to manufacturing) milestone, frantic last-minute engineering of Netscape 4's rendering engine was required in order to provide any support whatsoever for the W3C-mandated CSS—a necessarily slapdash effort that had long-term consequences for the quality of the browser, to say nothing of Netscape's viability.

Legacy Asset Inertia

During the era of the Web's fastest growth, standards were barely on the proverbial radar, and the cost of putting sites into production was high because the tools available at the time were quite primitive.

As a result of those adverse conditions, tremendous investments were made in poorly built web properties and the software that made them go. These properties continue to be nurtured because the cost of replacing them—measured in terms of institutional politics and capital investment—is seen as too high.

This phenomenon most strongly affects typical web developers in the area of third party content and solutions, particularly news publishing and advertising platforms.

Best Practices (and Lack Thereof)

Web shops and solo web developers can be found under a wide variety of institutional umbrellas: solo freelancers, specialist boutiques, large advertising agencies, mass media outlets, online businesses, medium-size businesses, information technology and information services departments of every imaginable size, and departments that have one or two developers fully responsible for the breadth of that unit's web presence. In addition, there are legions of do-it-yourself-ers, who can be undercapitalized, bloody minded, or both. Websites are built by all kinds of people, and everyone has different notions of what makes a website good or bad. That difference in judgment of quality

best practices.

An individual's most valuable web development qualification is neither her level of skill nor her degree of talent, but instead her ability to interact agreeably with teammates and others in order to be effective at her job, a quality frequently called "team fit." That dynamic is made still more complex by the fact that there is a high incidence of introvert personality traits amongst the population of professional web developers. Finally, the reluctance of many employers to take responsibility for their employees' ongoing skill development puts considerable drag on the momentum of median skills growth— sometimes to the point of eliminating that momentum completely.

As you can imagine, such an environment can result in wildly differing opinions regarding good and bad.

Strict Constructionism

The most passionate dispute that many developers have with the prospect of standards compliance is that it's an all-or-nothing affair. The most visible requirement of standards compliance is valid markup, which is too often impossible to publish because of the many challenges explained earlier.

In addition to meeting the extreme challenge of genuine standards compliance, well intentioned development teams must tolerate the rather shrill and morally superior attitude of many self-styled standards advocates, and the result is a large cadre of professionals who couldn't possibly care less about standards compliance.

Taking the Middle Road: Standards-Friendliness

Even when work product can't stand up perfectly to the test of validation, the good intentions that underlie web standards remain relevant.

The primary goal of standards-friendliness is to allow iterative enhancement of work product, a significant aspect of which is forward compatibility. Standards-friendliness leads to syntactically correct markup in good source order that strictly enforces separation of structure from presentation, while minimizing the amount of time spent struggling with the minutiae of validation. While this practice concedes more to adverse circumstances than many web standards advocates feel is appropriate—since after all, the project sponsor is the one who signs the paychecks—it still yields many benefits.

Benefits of Standards-Friendliness

What do you get in return for the extra effort of a standards-friendly approach?

Standards-friendly deliverables follow the Pareto (80/20) Rule, creating optimal benefit for the amount of time invested

The alternatives are unmaintainable assets, lack of scalability, and inflexibility. Such outcomes increase delivery times over the long term, since there's little in standards-ignorant work product on which to base modular assets. *Accessible assets become much easier to create*

When the source order of content is easily human-readable and developers use appropriate elements—especially lists, in the case of navigation functionality—the result presents few if any challenges for users of assistive technology.

Users of alternative media also benefit from these accessibility improvements

Standards-friendly development offers the best opportunities to apply separate presentation layers to content on a per-media (e.g., screen, print, mobile) basis without being forced to create multiple instances of the same content, saving huge amounts of time and other resources.

Data becomes more portable

When datastores are designed in the spirit of standards-friendliness, their contents can be transformed for use in other information systems, perhaps even systems that aren't web-based. Microformats advocates are especially ebullient about this benefit.

Rules of Standards-Friendly Development

Following some basic rules of standards-friendly development can simplify creating and developing your sites in the long run:

1. Avoid presentation-oriented markup (especially inline style attributes) at all reasonable cost.
2. Keep the source order of page content easily human-readable at all times.
3. Overbuild markup relating to overall document structure, but otherwise keep markup to the minimum required by the circumstances.
4. Relegate the use of table markup to data presentation *only*.
5. Ensure that all elements are properly closed, nested, populated, and supplied with required attributes; relegate all other validity concerns to a lower priority.
6. If possible, assign a distinctive id or class (or even one of each) to each body element within a site's scope, to account for edge cases.
7. Give all ids and classes names that are driven by context, rather than presentation. If you find yourself unable to follow this rule, you're probably breaking Rule #1;

46 | Chapter 4: Developing a Healthy Relationship with Standards

if you're able to follow it, you're probably doing a good job of following Rules #2 and #3.

8. Always use lists and headings where called for, even though it can be challenging to put the latter inside the former.
9. Relegate all images intended to provide design accents to the CSS background image property.
10. Draft separate stylesheets for Internet Explorer, limited to the scope of addressing layout bugs.

Like all rules, these have their exceptions—and because it de-emphasizes the value of *completely* valid markup, Rule #5 breaks a few broader rules itself.

The guiding principle of these practices is not *perfection* but *practicality*: it's often impossible to maintain perfect control over the entirety of a complex site, once poor third party content and other flaws are taken into account.

